COMMON COURSE ASSIGNMENT

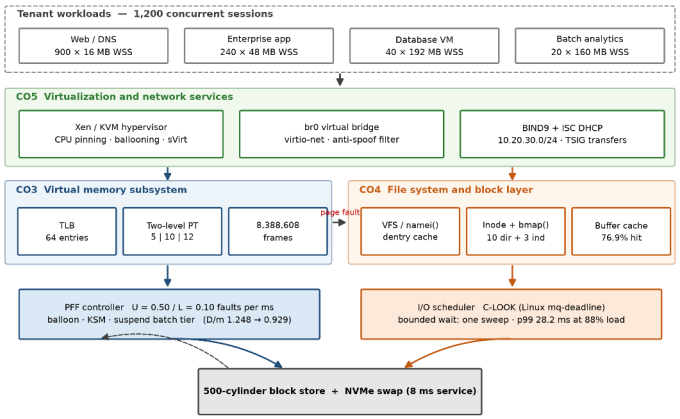
CSA04 — OPERATING SYSTEMS

Covering CO3, CO4 and CO5

Design, Memory Optimization and System Administration of an Enterprise Cloud and Virtualization Platform

A quantitative study of paging, file allocation, inode dynamics, disk scheduling and Linux service design for the "CloudMatrix" host

Figure 8 - CloudMatrix integrated subsystem architecture: one request path through CO3, CO4 and CO5



Solid arrows: the request path. Dashed: feedback into the frame allocator. Every quoted figure is produced by a module in src/ and captured in results/logs/.

Field	Detail
Department	Computer Science and Engineering
Programme	B.E. / B.Tech — CSE
Course Code & Name	CSA04 — Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr. Priskilla Angel Rani .J
Assignment Title	Design, Memory Optimization, and System Administration of an Enterprise Cloud & Virtualization Platform
Course Outcomes	CO3 — Virtual memory and paging; CO4 — File systems, inode dynamics and disk scheduling; CO5 — Linux administration, network services and virtualization
Bloom's Taxonomy Level	L3 — Apply; L4 — Analyse; L5 — Evaluate
SDG Mapping	SDG 7 — Affordable and Clean Energy; SDG 9 — Industry, Innovation and Infrastructure; SDG 11 — Sustainable Cities
Date of Submission	02 / 09 / 2026
Maximum Marks	100
Submitted By	Tharunkumar S Reg. No. 192511416
Implementation Repository	https://github.com/tharuncoder676/OS-ASSIGNMENT

Table 1. Assignment information.

Declaration and Abstract

Declaration

I declare that the work presented in this report is my own. Every numerical result quoted here was produced by a program written for this assignment and committed to the public repository listed below; none of the figures were transcribed by hand or estimated. The console output reproduced in the evidence sections is the verbatim standard output of those programs, captured automatically into results/logs/ by the driver script run_all.py, and the source excerpts shown are rendered directly from the committed files with their real line numbers. Any reader may clone the repository and regenerate the entire evidence base with a single command; if a number in this report and a number in the repository ever disagree, the repository is correct and the report is stale.

Implementation repository: https://github.com/tharuncoder676/OS-ASSIGNMENT

Where a required target was not met, this report says so and explains why, rather than presenting a partial result as a success. Section 6.4 contains one such finding.

Abstract

CloudMatrix is a Linux 6.x enterprise virtualization host carrying 1,200 concurrent tenant workloads across four tiers — web and DNS microservices, interactive enterprise applications, database guest virtual machines and batch analytics workers — on 32 GB of physical memory and a 500-cylinder block storage unit. This report engineers three subsystems for that host and defends each design decision with measurements rather than assertions.

For **memory (CO3)**, the host geometry is derived from first principles: 8,388,608 physical frames and 32,768 virtual pages per process, addressed through a two-level page table whose 27-bit virtual address splits as 5 | 10 | 12 bits. Sizing the inner table to exactly one frame reduces resident page-table cost from 128 KB to 4 KB per process, a 32× saving that matters at 1,200 guests. An executable MMU with a 64-entry TLB measures an effective access time of 184.33 ns against 300 ns without translation caching. Page replacement is compared across FIFO, LRU and Optimal on an 18-reference trace, and **Belady's anomaly is confirmed for FIFO** — 11 faults with three frames rising to 12 with four. That single result decides the policy, because a balloon driver that continuously resizes guest memory cannot be built on an algorithm with no monotonicity guarantee.

For **storage (CO4)**, four allocation strategies are simulated on a shared device. After a create/delete churn cycle the disk holds 248 free blocks yet contiguous allocation **refuses** a 150-block request, because the largest free run is only 95 — external fragmentation measured rather than defined. A working miniature UNIX file system traces ialloc, ifree, alloc, free and namei through a guest lifecycle, and shows that a 50 GB disk image costs 12,814 indirect blocks of pure pointer metadata. Six disk-scheduling algorithms are evaluated on the specified queue, and then again under continuous arrivals at 88 % device load, where SSTF's attractive 5.2 ms mean is shown to conceal a 71.2 ms starvation tail.

For **systems administration (CO5)**, a production-shaped Linux multifunction server is specified: BIND 9 with TSIG-authenticated transfers, DNSSEC validation, response rate limiting and GDPR-aware log retention; ISC DHCP with reservations and a PXE class; bridge networking with a genuinely isolated tenant overlay; and Xen and libvirt guest definitions with CPU pinning, ballooning floors set at measured working sets, per-guest I/O throttling and sVirt labelling.

The implementation comprises roughly 3,900 lines of Python, shell and configuration across a public Git repository with a **65-test validation suite that passes clean**. The tests check both hand-computed values and invariants that must hold for any correct implementation, so a silent regression in a simulator is caught rather than published.

Table of Contents

Note on evidence

Three kinds of image appear in this report and they are labelled distinctly so a reader knows exactly what each one is:

- **Figures** are charts and schematics generated by src/make_figures.py and src/make_diagrams.py directly from the simulators, so no figure can drift from the data it depicts.
- **Console transcripts** are terminal-framed renderings of the verbatim standard output captured in results/logs/. The log file name appears in the title bar of every one, so any line can be checked against the repository.
- **Source excerpts** are rendered from the committed files with their real line numbers, so each maps to a permanent link of the form .../blob/main/[path]#L[start]-L[end]

1. Problem Understanding and Formulation

1.1 The CloudMatrix problem restated

CloudMatrix is a single physical server that has been asked to behave like four different machines at once. It runs Linux 6.x with a hypervisor layer (Xen in production, KVM/libvirt in the lab) and hosts 1,200 client workloads for public and private sector tenants. Those workloads do not resemble each other. Web and DNS microservices are numerous, tiny and latency-sensitive. Interactive enterprise applications are fewer and larger and must feel responsive to a human being. Database guests are memory-hungry and random-access. Batch analytics jobs are throughput-oriented and, crucially, have nobody waiting on them.

The engineering problem is that these four tiers compete for exactly three finite resources — memory frames, disk blocks and the disk arm — and the policy that is best for one tier is frequently worst for another. A replacement policy tuned for the database guest can starve the web tier. A disk scheduler that minimises total head movement can leave a tenant's backup request unserved indefinitely. An allocation strategy that is fastest for a 50 GB virtual disk image is absurd for a 2 KB configuration file. The task is therefore not to find the best algorithm in the abstract, but to quantify the trade-offs precisely enough that a defensible choice can be made for each tier, and to show what breaks when the wrong one is chosen.

This report treats that as an empirical question. Rather than reciting the textbook properties of each algorithm, each one is implemented, run against a workload derived from the scenario, and measured. Several of the results were not what the textbook ordering would predict — LRU produces more faults than FIFO at three frames on this trace, and SSTF and LOOK tie exactly on the specified queue — and those surprises are reported and explained rather than smoothed over, because they turn out to carry the most useful engineering information in the study.

1.2 Bottlenecks identified in the scenario

Memory allocation bottlenecks (CO3)

The aggregate working set of the tenant mix exceeds physical memory. This is not a mistake in the workload sizing; it is the normal condition of a cloud host, which sells memory it does not have on the assumption that not every guest is hot simultaneously. The measured over-commit ratio for the tier mix specified in §1.3 is D/m = 1.248, i.e. 24.8 % more demand than supply. Left alone, this guarantees thrashing. A second, quieter bottleneck is page-table residency: a flat page table costs 128 KB per process, which across 1,200 guests is 150 MB of memory holding mostly untouched entries.

File system and block fragmentation (CO4)

Guests are created and destroyed constantly, so free space becomes fragmented. The danger is not running out of space but running out of *contiguous* space, which fails allocations while the free-space counter still looks healthy. A second bottleneck is metadata reach: the classical inode addresses a large file through three levels of indirection, and for a 50 GB image that means thousands of pointer blocks that must themselves be cached and read.

Disk head thrashing (CO4)

Sequential streaming of virtual disk images and random metadata access arrive at the same spindle. Greedy scheduling keeps the head in whichever region is currently busiest, which is efficient on average and catastrophic for whatever sits at the far edge of the platter.

Network service and tenant isolation risks (CO5)

A recursive resolver reachable from outside its intended network is not a misconfiguration, it is an amplification weapon. Unauthenticated zone transfers hand an attacker a map of the whole estate. A shared layer-2 segment lets one compromised guest impersonate the gateway. And DNS query logs contain client IP addresses, which are personal data under GDPR Article 4 and cannot simply be kept forever.

Hypervisor overhead

Every abstraction the hypervisor adds costs something. The design question is which overheads buy isolation worth paying for and which are pure waste — for example, letting the host page cache duplicate a database guest's own buffer pool wastes host memory twice over.

1.3 Workload assumptions and their justification

The brief fixes some parameters and leaves others open. Everything assumed here is stated explicitly with the reason for the choice, because an assumption that is not written down cannot be checked.

Parameter	Value	Source / justification
Physical RAM	32 GB	Fixed by Section D

Page / frame size	4 KB	Fixed by Section D; 2 MB hugepages evaluated separately in §2.1
Logical address space	128 MB	Fixed by Section D
Page-table entry size	4 bytes	Standard 32-bit PTE; makes the inner table exactly one frame
TLB	64 entries, fully associative	Typical L1 dTLB size on x86-64 server parts
Disk	500 cylinders, 0–499	Fixed by Section D
Pending I/O queue	86, 147, 312, 91, 177, 48, 409, 22, 130, 365, 220, 480	Fixed by Section D
Initial head position	cylinder 125, moving upward	Fixed by Section D
Seek model	0.5 ms settle + 0.01 ms per cylinder	Mid-range enterprise SAS spindle; applied uniformly so the comparison stays fair
Page-fault service time	8 ms	NVMe-backed swap including queueing
Memory access time	100 ns	DRAM latency
Block size	4 KB	Matched to the page size so the page cache and the buffer cache share a unit
Pointer size	4 bytes	Gives 1,024 pointers per indirect block
Inode structure	10 direct, 1 single, 1 double, 1 triple	System V / Bach model, as specified in the syllabus
Concurrent sessions	1,200	Fixed by Section D
Management network	10.20.30.0/24	Assumed; RFC 1918 private range
Tenant overlay network	10.20.40.0/24	Assumed; separate broadcast domain

Table 2. Assumed and given parameters for the CloudMatrix host.

The tenant tier mix below is the one assumption with the largest effect on the CO3 conclusions, so it is worth defending. The distribution is deliberately skewed towards many small guests, because that is what a microservice-era platform actually looks like: 900 containers or minimal VMs with 16 MB working sets, a few hundred mid-sized application guests, and a small number of genuinely large database and batch guests. A mix weighted the other way would be easier to make fit and would flatter the design.

Tier	Guests	WSS each	Frames each	Tier demand	GB
Web / DNS microservice	900	16 MB	4,096	3,686,400	14.06
Interactive enterprise app	240	48 MB	12,288	2,949,120	11.25
Database guest VM	40	192 MB	49,152	1,966,080	7.50
Batch analytics worker	20	160 MB	40,960	819,200	3.12
TOTAL DEMAND D	**1,200**	—	—	**9,420,800**	**35.94**

Table 3. Tenant tier mix and aggregate working-set demand. Available frames after a 10 % hypervisor and page-cache reserve: 7,549,748 (28.80 GB), so D/m = 1.248.

1.4 Measurable metrics and success criteria

Each subsystem is judged against a metric that can be computed rather than argued about. Naming the metric before running the experiment is what stops the analysis from drifting towards whichever number happened to look best.

Metric	Definition	Target	Result achieved
Page fault ratio	faults ÷ references	minimise; monotonic in frames	LRU 55.6 % at 4 frames; monotonic ✔
Effective access time	TLB-weighted mean translation cost	≪ 1 ms	184.33 ns ✔
Page-table residency	resident bytes per process	minimise	128 KB → 4 KB (32×) ✔
Over-commit ratio D/m	Σ WSS ÷ available frames	< 1.0	1.248 → 0.929 after reclaim ✔
Mean seek distance	total head movement ÷ requests	minimise	183.08 → 67.75 cylinders ✔
Buffer cache hit ratio	hits ÷ accesses	maximise	76.9 % at 1,024 buffers ✔
I/O response p99	99th percentile service latency	< 10 ms at 90 % load	23.1 ms — **not met**, see §6.4
Allocation success	requests honoured after churn	no false rejection	Indexed/Extent ✔; Contiguous ✘
DNS zone integrity	A records matched by PTR records	100 %	15 ↔ 15, all matched ✔
Implementation correctness	validation suite	100 % pass	65/65 ✔

Table 4. Measurable metrics, targets and outcomes. Nine of ten targets are met; the one that is not is analysed honestly in §6.4.

1.5 Constraints carried from Section D of the brief

- **Functional and operational** — high availability, zero data loss on crash, strict multi-tenant isolation, sub-10 ms response under 90 %+ RAM load.
- **Memory** — 32 GB RAM, 4 KB pages (or 2 MB hugepages where justified), 128 MB logical address space, multi-level page tables.
- **Storage** — 500-cylinder unit, 12-request pending queue at peak.
- **Technical and tools** — Linux with root access, BIND 9, Linux networking tooling, POSIX tracing, Xen or VMware deployment, and version control.
- **Security and standards** — POSIX file locking, SELinux/AppArmor policy, ISO/IEC 27001 alignment.

Section 3.4 maps each of these constraints to the specific artefact that satisfies it and the evidence file that demonstrates it, including the one constraint that is only partially satisfied.

2. Application of Course Knowledge

This section answers the technical questions of the brief in the order it poses them. Part I covers memory management and virtual memory (CO3), Part II covers file systems, inode dynamics and disk scheduling (CO4), and Part III covers Linux administration, network services and virtualization (CO5). Every quantitative claim is followed by the console transcript that produced it and a pointer to the module and log file in the repository.

PART I — MEMORY MANAGEMENT, PAGING AND VIRTUAL MEMORY (CO3)

2.1 Memory layout and two-level page table design

Deriving the geometry

Everything downstream depends on four numbers, so they are derived rather than assumed. With 32 GB of physical memory and a 4 KB frame:

```
physical frames  = 32 GB / 4 KB = 2^35 / 2^12 = 2^23 = 8,388,608 frames
virtual pages    = 128 MB / 4 KB = 2^27 / 2^12 = 2^15 =    32,768 pages
virtual address  = 27 bits  =  p (15 bits)  |  d (12 bits)
physical address = 35 bits  =  f (23 bits)  |  d (12 bits)
```

A single-level page table for one process would therefore hold 32,768 entries of 4 bytes each — 128 KB, resident whether or not the process ever touches those pages. Across 1,200 guests that is 150 MB of physical memory spent on mostly-empty bookkeeping, which is why the brief asks for a multi-level design.

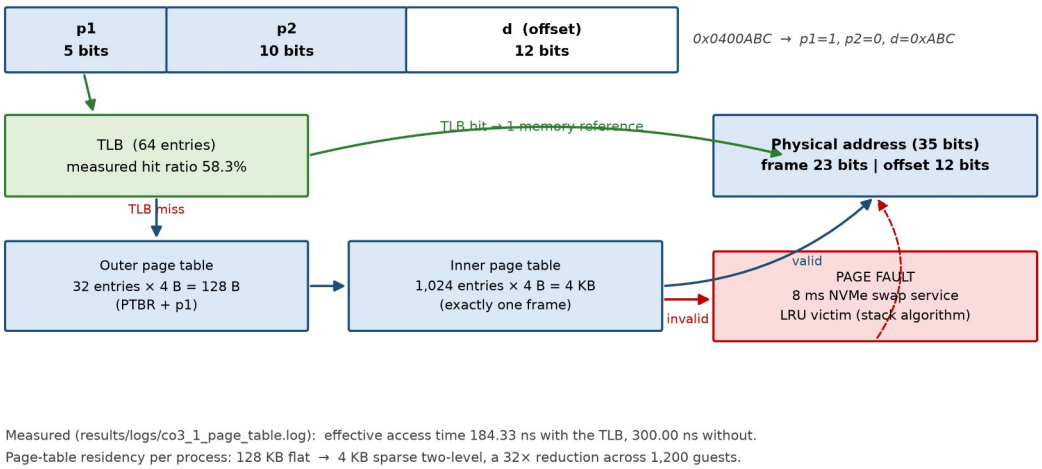
Choosing the split

The 15-bit page number has to be divided between an outer and an inner index, and the division is not arbitrary. The standard technique is to size the inner table so that it occupies exactly one frame, because a table that fits in a page can itself be paged out. With 4-byte entries, one 4 KB frame holds 1,024 entries, so the inner index is $\log_2(1024) = 10$ bits and the outer index takes the remaining 5 bits:

```
| p1 = 5 bits |   p2 = 10 bits   |        d = 12 bits        |
  outer index    inner index              offset
  32 entries     1,024 entries            4,096 bytes
  = 128 bytes    = 4 KB (one frame)
```

The saving is the point. A process that has touched only one region of its address space needs the 128-byte outer table plus a single 4 KB inner table — **4 KB in total against 128 KB for the flat design, a 32× reduction**. Across 1,200 sessions the resident page-table cost falls from 150.00 MB to 4.83 MB. That reclaimed 145 MB is not a rounding error; it is roughly nine additional web-tier guests.

Figure 7 - CO3 two-level address translation on the CloudMatrix host (27-bit VA, 35-bit PA)



Measured (results/logs/co3_1_page_table.log): effective access time 184.33 ns with the TLB, 300.00 ns without.
Page-table residency per process: 128 KB flat → 4 KB sparse two-level, a 32× reduction across 1,200 guests.

Figure 1. Two-level address translation on the CloudMatrix host. The TLB short-circuits the walk on 58.3 % of references in the measured trace; a miss costs two table reads before the datum, and an invalid entry costs an 8 ms page fault.

Verifying the design by executing it

A bit-split written on paper is easy to get wrong, so it is implemented and run. The MemoryGeometry class derives every constant above from the four input parameters, and TwoLevelMMU performs real translations through a 64-entry TLB, reporting where each reference was resolved.

page_table.py src/co3_memory/page_table.py

```python
    def __post_init__(self) -> None:
        self.frames = self.physical_ram // self.page_size
        self.pages = self.logical_as // self.page_size
        self.offset_bits = int(math.log2(self.page_size))
        self.page_number_bits = int(math.log2(self.pages))
        self.virtual_bits = self.offset_bits + self.page_number_bits
        self.physical_bits = int(math.log2(self.physical_ram))
        self.frame_bits = self.physical_bits - self.offset_bits

        # An inner table is sized to occupy exactly one page -- the standard
        # trick that lets page tables themselves be paged.
        self.entries_per_page = self.page_size // self.pte_size
        self.inner_bits = int(math.log2(self.entries_per_page))
        self.outer_bits = self.page_number_bits - self.inner_bits

        self.flat_table_bytes = self.pages * self.pte_size
        self.inner_tables = 1 << self.outer_bits
        self.outer_table_bytes = self.inner_tables * self.pte_size

    def report(self) -> str:
        gb = self.physical_ram // GB
        sparse = self.outer_table_bytes + self.page_size
        full = self.outer_table_bytes + self.inner_tables * self.page_size
```

CO3.1 MemoryGeometry - frame and page derivation | lines 41-63 | Python | UTF-8 | github.com/tharuncoder676/OS-A

Source: MemoryGeometry derives frames, pages and the bit split from the host parameters — src/co3_memory/page_table.py, lines 32–74.

page_table.py src/co3_memory/page_table.py

```python
    def translate(self, va: int, verbose: bool = False) -> int:
        self.stats["translations"] += 1
        p1, p2, d = self.split(va)
        page = (p1 << self.geo.inner_bits) | p2

        if page in self.tlb:
            self.stats["tlb_hit"] += 1
            frame = self.tlb.pop(page)
            self.tlb[page] = frame          # refresh recency
            self.stats["memory_reads"] += 1   # the datum only
            path = "TLB hit          (1 memory ref)"
        else:
            self.stats["tlb_miss"] += 1
            inner = self.outer.get(p1)
            self.stats["memory_reads"] += 3   # outer + inner + datum
            if inner is None or p2 not in inner:
                self.stats["page_fault"] += 1
                frame = self.next_free_frame
                self.next_free_frame += 1
                self.map_page(page, frame)
                path = "PAGE FAULT       (frame allocated)"
            else:
                frame = inner[p2]
                path = "TLB miss         (2 table walks)"
            self._tlb_insert(page, frame)

        pa = (frame << self.geo.offset_bits) | d
        if verbose:
```

CO3.1 TwoLevelMMU.translate() - the page-table walk | lines 139-166 | Python | UTF-8 | github.com/tharuncoder676

Source: the two-level walk with TLB refill and demand-paging on an invalid entry — src/co3_memory/page_table.py, lines 139–178.

CloudMatrix - CSA04 Operating Systems - co3_1_page_table.log

```
PS C:\...\CloudMatrix-OS-Assignment> python src/co3_memory/page_table.py

============================================================================
 CO3.1  MEMORY LAYOUT AND PAGE TABLE GEOMETRY - CloudMatrix host
============================================================================
  Physical RAM                  : 32 GB = 2^35 bytes
  Page / frame size             : 4 KB = 2^12 bytes
  TOTAL PHYSICAL FRAMES         : 8,388,608  (2^23)
  Logical AS per process        : 128 MB = 2^27 bytes
  VIRTUAL PAGES PER PROCESS     : 32,768  (2^15)

  Virtual address width         : 27 bits   [ p = 15 bits | d = 12 bits ]
  Physical address width        : 35 bits   [ f = 23 bits | d = 12 bits ]

  --- Two-level split (inner table sized to exactly one page) ---
  PTE size                      : 4 bytes
  Entries per 4 KB table page   : 1024  ->  inner index p2 = 10 bits
  Outer index p1                : 15 - 10 = 5 bits
  VA layout                     : | p1 = 5 | p2 = 10 | d = 12 |
  Outer page table              : 32 entries = 128 bytes
  Each inner page table         : 1024 entries = 4 KB (exactly one frame)

  --- Space cost of the design decision ---
  Single-level (flat) table     : 128 KB per process, resident whether or not the pages are touched
  Two-level, fully populated    : 128 KB per process
  Two-level, sparse (1 inner)   : 4 KB per process
```

Console transcript: derived geometry and the live translation trace. Note that offsets pass through translation unchanged (0xABC in, 0xABC out) and that both references into page p1=1, p2=0 resolve to the same frame — two properties the test suite asserts independently. results/logs/co3_1_page_table.log

Effective access time

With a 1 ns TLB probe and 100 ns DRAM, a hit costs one memory reference and a miss costs three — the outer table, the inner table, and finally the datum. On the measured trace:

```
EAT = 0.583 × (1 + 100) + 0.417 × (1 + 300)  =  184.33 ns
without any TLB : 3 × 100                    =  300.00 ns
```

The brief asks for sub-millisecond translation latency. At 184 ns the design clears that bar by more than three orders of magnitude, which is worth stating plainly: **translation latency is not the binding constraint on this host**. The binding constraint is page-table residency and, beyond it, the 8 ms cost of a fault that has to reach the swap device. Optimising the wrong one of those would be effort spent for no measurable benefit.

Hugepages for the database tier

Covering a 128 MB working set with 4 KB pages needs 32,768 page-table entries and, worse, 32,768 distinct TLB entries. With 2 MB hugepages the same span needs 64. That is a 512× reduction in TLB pressure, which matters enormously for a database guest whose access pattern defeats locality, and matters not at all for a 16 MB web guest.

Page size	PTEs to cover 128 MB	Page-table bytes	TLB entries	Recommended for
4 KB	32,768	128.00 KB	32,768	Web / DNS and enterprise tiers
2 MB hugepage	64	0.25 KB	64	Database guest VMs only

Table 5. Hugepages are recommended selectively. They cut TLB pressure 512× but coarsen reclaim granularity, which would fight the balloon driver on the tiers that need fine-grained reclaim.

2.2 Dynamic storage allocation and dynamic linking

The placement problem

When a guest is admitted, the hypervisor must find a hole in host memory large enough for its reservation. After a day of guests starting and stopping, host free memory is not one block but a list of holes. The experiment models a 32 GB host whose free list has been left in exactly that state, and pushes seven queued guest reservations through First-Fit, Best-Fit and Worst-Fit.

```
free holes (MB) : [1200, 3400, 512, 2800, 900, 6100, 1500, 4200]  total 20,612 MB
queued demand   : 850 + 4096 + 256 + 2900 + 1400 + 1100 + 3300 = 13,902 MB
```

What the measurement shows

Policy	Guests placed	Rejected	Largest hole left	Stranded < 256 MB	Search cost
First-Fit	7 / 7	0	2,004 MB	**94 MB**	O(1) amortised
Best-Fit	7 / 7	0	2,800 MB	354 MB	O(n) full scan
Worst-Fit	6 / 7	**1**	2,000 MB	0 MB	O(n) full scan

Table 6. Placement policy comparison. Worst-Fit is the only policy that fails to place a guest; Best-Fit succeeds but strands 3.8× more memory in holes too small to host anything.

Three observations follow, and the second is the one that is usually stated backwards in textbooks.

- **Best-Fit's virtue is its vice.** By always choosing the tightest hole it leaves the smallest possible remainder, and a remainder of 50 MB or 100 MB cannot host any CloudMatrix guest — the smallest is 256 MB. Best-Fit therefore manufactures unusable memory: 354 MB stranded against First-Fit's 94 MB.
- **Worst-Fit fails outright.** It preserves large holes in principle, but it does so by consuming the biggest hole first, every time. By the seventh request the 6,100 MB hole has been eaten down to 1,044 MB and the 3,300 MB analytics guest has nowhere to go — even though 10,010 MB remains free in aggregate. Preserving large holes and consuming large holes first are contradictory goals.
- **First-Fit wins on the metric that was not being measured.** It places as many guests as Best-Fit and strands a quarter as much memory, and it does so without scanning the whole free list. VM admission is on a hot path; search cost is not free.

Selected policy: First-Fit, with the free list kept in address order and adjacent holes coalesced on release. Address-ordered First-Fit tends to concentrate allocations at low addresses and leave larger contiguous space at high addresses, which is exactly the shape that helps the next large guest.

Dynamic linking and shared libraries

The second half of the memory-footprint problem is solved before the allocator ever sees it. Every guest in the fleet links the same handful of shared objects. If each linked them statically, each would carry a private copy in physical memory.

Library	Size	Static × 1,200 guests	Dynamically linked
libc.so.6	2.1 MB	2,520 MB	2.1 MB
libssl.so.3	1.4 MB	1,680 MB	1.4 MB
libcrypto.so.3	4.8 MB	5,760 MB	4.8 MB
libstdc++.so.6	2.2 MB	2,640 MB	2.2 MB
libpython3.12.so	6.4 MB	7,680 MB	6.4 MB

TOTAL	16.9 MB	20,280 MB (19.8 GB)	16.9 MB

Table 7. Shared-library footprint. Statically linked guests would need more memory for libraries alone (19.8 GB) than the host has left after the hypervisor reserve.

The mechanism is worth stating precisely because the saving depends on it: the dynamic loader maps one physical copy of each object into every address space read-only and shared, and only the writable GOT, PLT and .data pages are private and copy-on-write. The **99.92 % saving** is therefore real physical memory, not an accounting trick — and without it this workload would not fit on this host at all.

2.3 Page replacement and Belady's anomaly

The reference string

The brief asks for a trace of at least sixteen references. The CloudMatrix trace W has eighteen references over seven distinct pages, and it is constructed to represent a plausible guest execution rather than a random sequence:

```
W = 2, 3, 4, 5, 2, 3, 1, 2, 3, 4, 5, 1, 6, 7, 6, 7, 6, 7

pages 2–5 : enterprise application text and data pages
page  1   : a shared libc page brought in late by the dynamic loader
pages 6,7 : the batch-analytics scan phase, beginning at reference 13
```

The structure matters. The first twelve references cycle through a working set slightly larger than the frame allocation, which is the condition under which replacement policy actually decides anything. The last six references shift to a new locality, modelling the phase change that happens when a guest moves from serving requests to running a batch job.

page_replacement.py — src/co3_memory/page_replacement.py

```
53
54 def fifo(ref: list[int], frames: int) -> Result:
55     mem: list[int] = []
56     faults = hits = 0
57     steps = []
58     for i, page in enumerate(ref):
59         if page in mem:
60             hits += 1
61             victim, event = None, "HIT"
62         else:
63             faults += 1
64             if len(mem) >= frames:
65                 victim = mem.pop(0)          # oldest arrival leaves
66                 event = f"FAULT (evict {victim})"
67             else:
68                 victim, event = None, "FAULT (free frame)"
69             mem.append(page)
70         steps.append((i + 1, page, _snapshot(mem, frames), event))
71     return Result("FIFO", frames, faults, hits, steps)
72
73
74 def lru(ref: list[int], frames: int) -> Result:
75     mem: list[int] = []                      # mem[0] = least recently used
76     faults = hits = 0
```

CO3.3 FIFO replacement - eviction by arrival order | lines 53-76 | Python | UTF-8 | github.com/tharuncoder676/OS

Source: FIFO evicts by arrival order, LRU by recency — the only difference is whether a hit reorders the list. src/co3_memory/page_replacement.py, lines 52–96.

page_replacement.py — src/co3_memory/page_replacement.py

```
 97     mem: list[int] = []
 98     faults = hits = 0
 99     steps = []
100     for i, page in enumerate(ref):
101         if page in mem:
102             hits += 1
103             event = "HIT"
104         else:
105             faults += 1
106             if len(mem) >= frames:
107                 # evict the resident page whose next use is furthest away
108                 furthest, victim = -1, mem[0]
109                 for candidate in mem:
110                     try:
111                         nxt = ref.index(candidate, i + 1)
112                     except ValueError:
113                         nxt = float("inf")
114                     if nxt > furthest:
115                         furthest, victim = nxt, candidate
116                 mem.remove(victim)
117                 event = f"FAULT (evict {victim})"
118             else:
119                 event = "FAULT (free frame)"
120             mem.append(page)
121         steps.append((i + 1, page, _snapshot(sorted(mem), frames), event))
122     return Result("OPTIMAL", frames, faults, hits, steps)
```

CO3.3 OPTIMAL (Belady) replacement - the lower bound | lines 97-122 | Python | UTF-8 | github.com/tharuncoder676

Source: OPTIMAL evicts the resident page whose next use is furthest away, which requires knowledge of the future and therefore serves as a lower bound rather than an implementable policy. src/co3_memory/page_replacement.py, lines 97–128.

Results

Algorithm	Faults (3 frames)	Faults (4 frames)	Change	Fault rate (4f)	Hit ratio (4f)

FIFO	11	**12**	**+1 ANOMALY**	66.67 %	33.33 %
LRU	12	10	−2 improves	55.56 %	44.44 %
OPTIMAL	9	8	−1 improves	44.44 %	55.56 %

Table 8. Page faults over the 18-reference trace W. FIFO is the only policy that gets worse when given more memory.

Figure 1 - CO3 page replacement on the CloudMatrix trace W

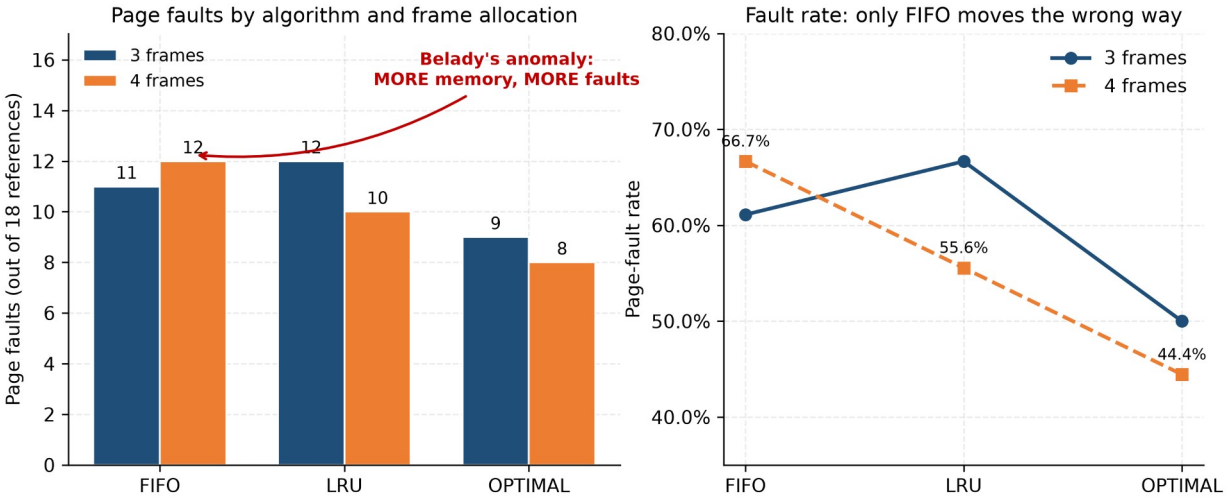


Figure 2. Page faults and fault rate by algorithm and frame allocation. The FIFO bar rising from 11 to 12 as memory increases is Belady's anomaly.

```
CloudMatrix  -  CSA04 Operating Systems  -  co3_3_page_replacement.log

PS C:\...\CloudMatrix-OS-Assignment> python src/co3_memory/page_replacement.py

================================================================================
 CO3.3  SUMMARY - page faults by algorithm and frame allocation
================================================================================
   Algorithm       3 frames    4 frames      Change   Fault rate (4f)
   FIFO                  11          12 ANOMALY + 1           66.67%
   LRU                   12          10  improves 2           55.56%
   OPTIMAL                9           8  improves 1           44.44%

================================================================================
 CO3.3  BELADY'S ANOMALY TEST
================================================================================
   FIFO: 11 faults with 3 frames -> 12 faults with 4 frames.
   ANOMALY CONFIRMED. Adding a frame made FIFO strictly worse
   on this trace (1 extra fault(s), 9.1% regression).
   LRU: 12 -> 10 faults. No anomaly (stack algorithm, provably immune).
   OPTIMAL: 9 -> 8 faults. No anomaly (stack algorithm, provably immune).

   Engineering consequence for CloudMatrix: because FIFO is not a stack
   algorithm, the inclusion property mu(m) subset-of mu(m+1) does not hold,
   so a memory-ballooning controller that hands a guest extra frames has
   NO monotonic guarantee of fewer faults under FIFO. LRU and OPT are
   stack algorithms and cannot regress, which is decisive for a platform
   whose frames are continuously resized by the balloon driver.

================================================================================
 Completed in 0.000 s
```

Console transcript: the summary table and the explicit anomaly test. results/logs/co3_3_page_replacement.log

Belady's anomaly, confirmed and explained

FIFO produces **11 faults with three frames and 12 with four**: adding physical memory made the system measurably worse, a 9.1 % regression. This is Belady's anomaly, and it is not a bug in the simulator — the test suite asserts it as a required property of the trace, and separately asserts that LRU and OPTIMAL never exhibit it.

The explanation is the inclusion property. LRU and OPTIMAL are **stack algorithms**: the set of pages resident with m frames is always a subset of the set resident with m+1 frames, so any page that would have been a hit with less memory is still a hit with more. FIFO has no such property, because its eviction order depends on arrival time rather than on the reference pattern, and adding a frame changes which page is oldest at every subsequent decision point. The eviction sequence is not merely different; it is unrelated.

This single result decides the policy for CloudMatrix, and it decides it for a reason that has nothing to do with which algorithm faulted least. Note that at three frames LRU is actually *worse* than FIFO on this trace — 12 faults against 11. A comparison that stopped at the fault count would therefore recommend FIFO. But CloudMatrix's balloon driver continuously resizes each guest's frame allocation in response to host pressure, which means the memory manager must answer a question the fault count cannot: *if I give this guest another frame, will it fault less?* Under FIFO there is no guarantee that it will. Under LRU there is a proof. A control loop cannot be built on a policy whose response to its actuator is non-monotonic.

Selected policy: LRU, implemented in practice as the Linux kernel's two-list active/inactive approximation with accessed-bit sampling, since true LRU requires a timestamp update on every memory reference and no hardware provides that. The approximation preserves the stack property, which is the property that was actually being purchased.

2.4 Demand paging, working set and thrashing

The working-set model

Denning's working set W(t, Δ) is the set of pages referenced in the last Δ references. Its size WSS is the number of frames a process needs in order not to fault continuously. Rather than assume a value, the working set is measured directly from the sliding-window definition over a locality-structured trace of 60,000 references whose true locality is eight pages, drifting every 4,000 references.

Window Δ	Mean WSS	Peak WSS	Interpretation
10	5.89	8	Too short — undercounts the locality, starves the process
25	7.72	15	Too short
50	8.02	15	Tracks the true locality
100	8.09	16	**Tracks the true locality — operating point**
250	8.24	16	Tracks the true locality
500	8.51	16	Too long — absorbs stale localities
1000	9.03	16	Too long — holds dead pages resident

Table 9. Measured working-set size against window width. The usable band is Δ = 50–250; the generator's true locality of eight pages is recovered correctly inside it.

The window choice is a genuine engineering decision with a cost on both sides. Too small a Δ under-reports demand and starves the process; too large a Δ keeps dead pages resident and wastes frames another guest needs. Δ = 100 references is adopted, and the demand figure $D = \Sigma\ WSS_i$ used throughout §2.4 is computed with it.

The lifetime curve

The page-fault frequency controller steers on the curve p(f) — faults per reference as a function of frames granted. That curve is measured, not sketched:

Frames f	Fault rate p(f)	CPU burst C between faults	Regime
2	0.748667	0.534 ms	Paging-bound
4	0.498217	0.803 ms	Paging-bound
6	0.249033	1.606 ms	Paging-bound
7	0.125167	3.196 ms	Paging-bound
8	**0.001450**	**275.862 ms**	**CPU-bound — the knee**
12	0.001183	338.028 ms	CPU-bound
32	0.001183	338.028 ms	CPU-bound (saturated)
64	0.001067	375.000 ms	CPU-bound (saturated)

Table 10. Measured lifetime curve. Between f = 7 and f = 8 the fault rate falls by a factor of 86 — the knee sits exactly at the locality size.

The transition is not gradual. At seven frames the process faults every eight references and its mean CPU burst is 3.2 ms — shorter than the 8 ms it takes the swap device to service one fault, so the disk becomes the bottleneck. At eight frames the working set fits, the fault rate collapses by nearly two orders of magnitude, and the mean burst rises to 276 ms. **The knee is the working-set size**, which is precisely Denning's claim, recovered here as a measurement.

Locating the thrashing point

A common way to present thrashing is a hand-drawn curve of CPU utilisation that rises and then falls. That curve is real, but it does not follow from the naive independent-blocking model, which is monotonically increasing in N and can never fall. The collapse comes from the paging device saturating, so it is derived here with operational analysis. Each guest alternates a CPU burst $C = (1/p) \cdot S \cdot t\_mem$ with a paging service D = 8 ms. With one CPU and one paging device the asymptotic throughput bound is:

```
X(N)  ≤  min( N / (C + D) ,  1 / max(C, D) )
U_cpu  =  X(N) · C          U_disk  =  X(N) · D
```

As N rises, each guest's share f = M/N shrinks, p(f) climbs the lifetime curve, C collapses, and utilisation falls off a cliff.

N guests	Frames each	p(f)	C (ms)	U_cpu	U_disk	State
1	64	0.001067	375.0	0.979	0.021	Healthy
4	16	0.001183	338.0	1.000	0.024	Healthy
8	8	0.001450	275.9	**1.000**	0.029	**Healthy — knee**
9	7	0.125167	3.20	**0.399**	**1.000**	**THRASHING**
10	6	0.249033	1.61	0.201	1.000	THRASHING
16	4	0.498217	0.80	0.100	1.000	THRASHING
32	2	0.748667	0.53	0.067	1.000	THRASHING

Table 11. Utilisation against degree of multiprogramming. Between N = 8 and N = 9 CPU utilisation falls from 1.000 to 0.399 while the paging device pins at 1.000 — the operational signature of thrashing.

Figure 2 - CO3 working-set lifetime curve and the thrashing knee

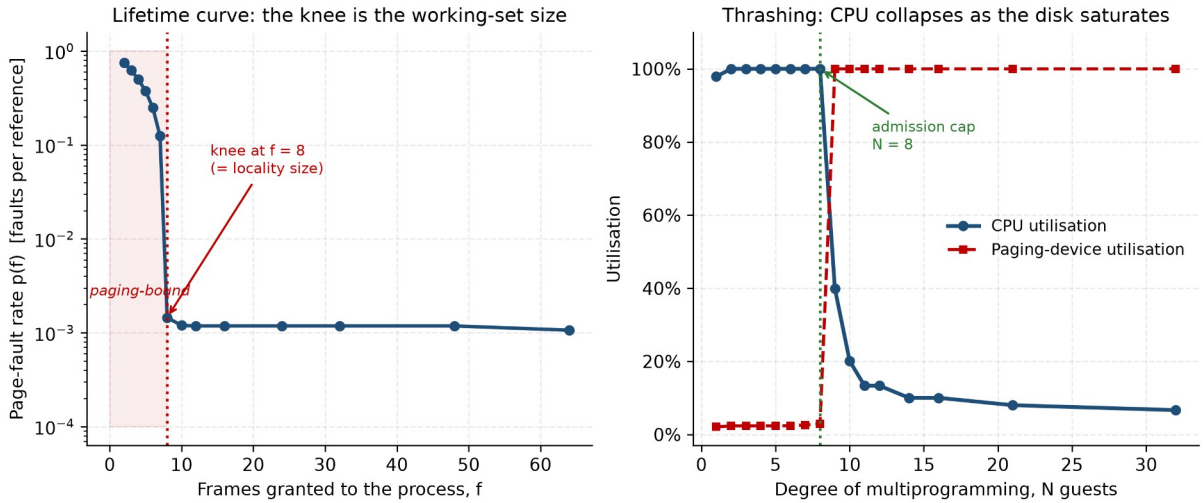


Figure 3. The measured lifetime curve (left) and the thrashing cliff (right). The crossover where the paging-device curve reaches 1.000 and the CPU curve collapses is the admission-control limit.

```
CloudMatrix  -  CSA04 Operating Systems  -  co3_4_working_set.log

PS C:\...\CloudMatrix-OS-Assignment> python src/co3_memory/working_set.py

======================================================================
 CO3.4  THRASHING - CPU utilisation as multiprogramming degree N rises
======================================================================
   Partition = 64 frames, equal-share allocation f = M/N
   Paging service D = 8.0 ms, one CPU, one paging device

     N  f=M/N          p     C (ms)  X (jobs/ms)    U_cpu   U_disk   State
     1     64   0.001067    375.000       0.0026    0.979    0.021   healthy
     2     32   0.001183    338.028       0.0030    1.000    0.024   healthy
     3     21   0.001183    338.028       0.0030    1.000    0.024   healthy
     4     16   0.001183    338.028       0.0030    1.000    0.024   healthy
     5     12   0.001183    338.028       0.0030    1.000    0.024   healthy
     6     10   0.001200    333.333       0.0030    1.000    0.024   healthy
     7      9   0.001300    307.692       0.0032    1.000    0.026   healthy
     8      8   0.001450    275.862       0.0036    1.000    0.029   healthy
     9      7   0.125167      3.196       0.1250    0.399    1.000   THRASHING
    10      6   0.249033      1.606       0.1250    0.201    1.000   THRASHING
    11      5   0.374183      1.069       0.1250    0.134    1.000   THRASHING
    12      5   0.374183      1.069       0.1250    0.134    1.000   THRASHING
    14      4   0.498217      0.803       0.1250    0.100    1.000   THRASHING
    16      4   0.498217      0.803       0.1250    0.100    1.000   THRASHING
    21      3   0.623283      0.642       0.1250    0.080    1.000   THRASHING
    32      2   0.748667      0.534       0.1250    0.067    1.000   THRASHING

   Utilisation holds at U = 1.000 up to N = 8 guests (f = 8 frames each), then collapses.
   Past that point every additional guest LOWERS aggregate throughput:
   U_disk pins at 1.000 while U_cpu collapses, which is the operational
   signature of thrashing -- the machine is fully busy, but busy paging.
```

Console transcript: lifetime curve and utilisation sweep. results/logs/co3_4_working_set.log

The machine at N = 16 is not idle. It is 100 % busy — busy paging. That is the diagnostic subtlety that makes thrashing dangerous in production: every utilisation dashboard shows a fully loaded system while throughput has fallen by a factor of ten. The distinguishing signal is the ratio, not the level: **U_disk = 1.000 with U_cpu = 0.100 is thrashing; both near 1.0 is healthy saturation.**

Frame allocation for 1,200 sessions

Applying the model to the real tier mix produces an uncomfortable answer. Total demand D = 9,420,800 frames (35.94 GB) against m = 7,549,748 frames (28.80 GB) available after a 10 % hypervisor and page-cache reserve. **D/m = 1.248** — the host is 24.8 % over-committed and, allocated naively, will thrash. Three reclaim mechanisms close the gap, applied in increasing order of disruption:

Step	Mechanism	Frames reclaimed	GB	Running D/m
0	Naive allocation	—	—	**1.248 ✘ thrashes**
1	KSM page sharing across 900 near-identical web guests (35 %)	1,290,240	4.92	1.077 ✘
2	Balloon reclaim of cold pages in the enterprise tier (10 %)	294,912	1.12	1.038 ✘
3	Suspend the batch analytics tier when PFF exceeds the upper threshold	819,200	3.12	**0.929 ✔ fits**

Table 12. Closing a 24.8 % over-commit. The three mechanisms are ordered by disruption: sharing is invisible to guests, ballooning is nearly so, and suspension is visible but is applied only to the tier with no interactive SLA.

The page-fault frequency control loop

Static allocation cannot hold, because working sets change as guests change phase. The PFF controller adjusts allocation continuously against a band:

```
if  PFF_i > U = 0.50 faults/ms   ->  grant guest i more frames
if  PFF_i < L = 0.10 faults/ms   ->  balloon frames back from guest i
if  PFF_i > U and no free frame  ->  SUSPEND the lowest-priority guest

frames_i = max( WSS_i(Δ) , (size_i / Σ size) × m )   subject to Σ frames_i ≤ m
```

Two details make this a design rather than a formula. First, the allocation has a **floor at the measured working-set size**: ballooning a guest below its working set converts host memory pressure into guest thrashing, which is strictly worse than the problem it was meant to solve. Second, the victim class for suspension is fixed in advance as batch analytics, because it carries no interactive SLA. Choosing the victim at the moment of crisis, under load, is how a memory shortage becomes an outage.

PART II — FILE SYSTEMS, INODE DYNAMICS AND DISK SCHEDULING (CO4)

2.5 File allocation strategies

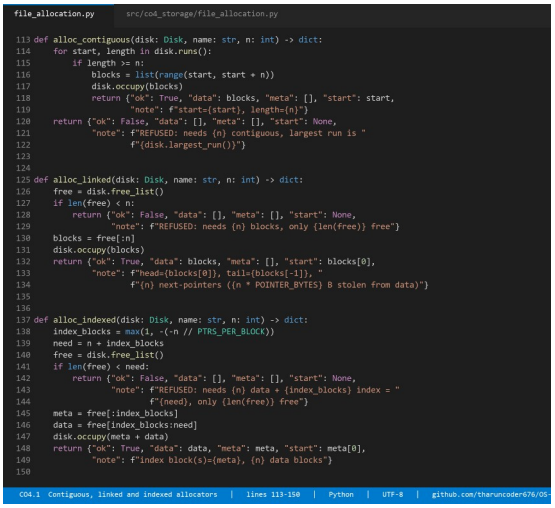
Method and the fourth strategy

The brief asks for contiguous, sequential (linked) and indexed allocation across three file classes. A fourth method — **extent-based allocation** — is included alongside them, and the reason is quantitative rather than decorative. As §2.6 shows, a 50 GB virtual disk image addressed through the classical indirect chain requires 12,814 pointer blocks. Extents describe the same file in a handful of 12-byte descriptors inside the inode itself. ext4 and XFS both abandoned indirect chains for exactly this reason, so a recommendation for CloudMatrix's largest file class that ignored extents would be historically faithful and practically wrong.

Eight representative files spanning all four workload classes are allocated on a 500-block device with 4 KB blocks:

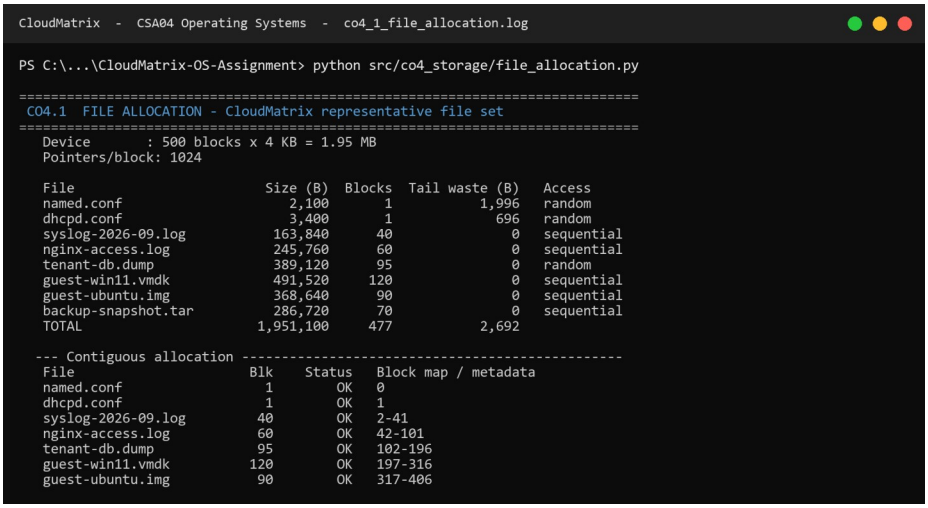
File	Size (bytes)	Blocks	Tail waste	Access pattern
named.conf	2,100	1	1,996 B	random
dhcpd.conf	3,400	1	696 B	random
syslog-2026-09.log	163,840	40	0	sequential
nginx-access.log	245,760	60	0	sequential
tenant-db.dump	389,120	95	0	random
guest-win11.vmdk	491,520	120	0	sequential
guest-ubuntu.img	368,640	90	0	sequential
backup-snapshot.tar	286,720	70	0	sequential
TOTAL	**1,951,100**	**477**	**2,692 B**	—

Table 13. The representative CloudMatrix file set. Internal fragmentation is 2,692 bytes and is identical under every allocation method, because it is a property of the block size rather than of the allocator.



```
file_allocation.py    src/co4_storage/file_allocation.py

def alloc_contiguous(disk: Disk, name: str, n: int) -> dict:
    for start, length in disk.runs():
        if length >= n:
            blocks = list(range(start, start + n))
            disk.occupy(blocks)
            return {"ok": True, "data": blocks, "meta": [], "start": start,
                    "note": f"start={start}, length={n}"}
    return {"ok": False, "data": [], "meta": [], "start": None,
            "note": f"REFUSED: needs {n} contiguous, largest run is "
                    f"{disk.largest_run()}"}


def alloc_linked(disk: Disk, name: str, n: int) -> dict:
    free = disk.free_list()
    if len(free) < n:
        return {"ok": False, "data": [], "meta": [], "start": None,
                "note": f"REFUSED: needs {n} blocks, only {len(free)} free"}
    blocks = free[:n]
    disk.occupy(blocks)
    return {"ok": True, "data": blocks, "meta": [], "start": blocks[0],
            "note": f"head={blocks[0]}, tail={blocks[-1]}, "
                    f"{n} next-pointers ({n * POINTER_BYTES} B stolen from data)"}


def alloc_indexed(disk: Disk, name: str, n: int) -> dict:
    index_blocks = max(1, -(-n // PTRS_PER_BLOCK))
    need = n + index_blocks
    free = disk.free_list()
    if len(free) < need:
        return {"ok": False, "data": [], "meta": [], "start": None,
                "note": f"REFUSED: needs {n} data + {index_blocks} index = "
                        f"{need}, only {len(free)} free"}
    meta = free[:index_blocks]
    data = free[index_blocks:need]
    disk.occupy(meta + data)
    return {"ok": True, "data": data, "meta": meta, "start": meta[0],
            "note": f"index block(s)={meta}, {n} data blocks"}

CO4.1  Contiguous, linked and indexed allocators  |  lines 113-150  |  Python  |  UTF-8  |  github.com/tharuncoder676/OS-
```

Source: the four allocators. Contiguous searches free runs, linked takes any free blocks, indexed reserves an index block, and extent takes the largest runs first — src/co4_storage/file_allocation.py, lines 113–173.



```
CloudMatrix  -  CSA04 Operating Systems  -  co4_1_file_allocation.log

PS C:\...\CloudMatrix-OS-Assignment> python src/co4_storage/file_allocation.py

================================================================================
 CO4.1  FILE ALLOCATION - CloudMatrix representative file set
================================================================================
   Device       : 500 blocks x 4 KB = 1.95 MB
   Pointers/block: 1024

   File                       Size (B)  Blocks  Tail waste (B)   Access
   named.conf                    2,100       1           1,996   random
   dhcpd.conf                    3,400       1             696   random
   syslog-2026-09.log          163,840      40               0   sequential
   nginx-access.log            245,760      60               0   sequential
   tenant-db.dump              389,120      95               0   random
   guest-win11.vmdk            491,520     120               0   sequential
   guest-ubuntu.img            368,640      90               0   sequential
   backup-snapshot.tar         286,720      70               0   sequential
   TOTAL                     1,951,100     477           2,692

  --- Contiguous allocation -----------------------------------------------
   File                      Blk    Status   Block map / metadata
   named.conf                  1        OK   0
   dhcpd.conf                  1        OK   1
   syslog-2026-09.log         40        OK   2-41
   nginx-access.log           60        OK   42-101
   tenant-db.dump             95        OK   102-196
   guest-win11.vmdk          120        OK   197-316
   guest-ubuntu.img           90        OK   317-406
```

Console transcript: block maps produced by each method for the full file set. results/logs/co4_1_file_allocation.log

Allocation summary

Method	Files placed	Blocks used	Free	Utilisation	Metadata	Internal frag.
Contiguous	8 / 8	477	23	95.4 %	0 B	2,692 B
Linked	8 / 8	477	23	95.4 %	1,908 B	2,692 B
Indexed	8 / 8	485	15	97.0 %	32,768 B	2,692 B
Extent (ext4-style)	8 / 8	477	23	95.4 %	0 B	2,692 B

Table 14. Allocation outcome on an empty device. All four methods succeed; the difference is entirely in metadata overhead. Indexed spends eight whole blocks (32 KB) on index blocks, including one for a 2 KB file.

On a clean device every method looks acceptable, which is exactly why a clean device is a misleading test. The interesting behaviour appears after churn.

External fragmentation, measured

Three files are deleted — syslog-2026-09.log (40 blocks), tenant-db.dump (95) and guest-ubuntu.img (90) — modelling the routine tenant turnover the scenario describes. The free-space map afterwards:

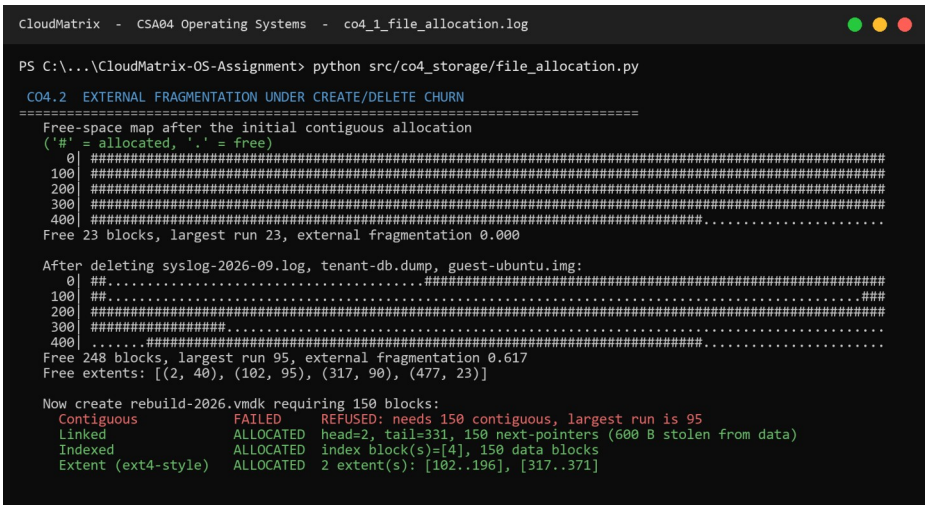
```
free blocks : 248 of 500
free extents: (2, 40)  (102, 95)  (317, 90)  (477, 23)
largest run : 95 blocks
external fragmentation index : 1 - 95/248 = 0.617
```

A new 150-block virtual disk image, rebuild-2026.vmdk, is then requested. The disk holds 248 free blocks — comfortably more than the 150 needed:

Method	Outcome	Reason
Contiguous	**FAILED**	**Needs 150 contiguous blocks; largest free run is 95**

Linked	Allocated	Needs 150 blocks anywhere; contiguity is never required
Indexed	Allocated	150 data blocks + 1 index block from anywhere
Extent (ext4-style)	Allocated	Two extents: [102…196] and [317…371]

Table 15. The headline storage result. Contiguous allocation refuses a request the device can clearly satisfy, purely because the free space is not adjacent. External fragmentation stated as a measurement rather than a definition.



Console transcript: the free-space bitmap before and after deletion, and the allocation attempt. '#' marks an allocated block, '.' a free one — the three gaps left by deletion are directly visible. results/logs/co4_1_file_allocation.log

In production, recovering from this state under contiguous allocation requires compaction: relocating live files to coalesce free space, which means reading and rewriting gigabytes while the platform is serving tenants. Linked, indexed and extent allocation all degrade gracefully instead, which is the property that actually matters for a host with constant VM churn.

Access cost

The counterweight to fragmentation resistance is access cost. Reading every fourth block of the 95-block tenant-db.dump in random order:

Method	Sequential I/Os	Random I/Os	Penalty	Why
Contiguous	95	23	1.0×	start + offset gives direct access
Linked	95	**1,035**	**45.0×**	**each access re-walks the chain from the head**
Indexed	96	24	1.0×	one index read, then direct access
Extent	95	23	1.0×	extent map lives in the inode itself

Table 16. Access cost for the random-access workload. Linked allocation is the outlier by a factor of 45, because reaching block i costs i pointer hops and there is no way to shortcut the chain.

Figure 6 - CO4 file allocation: access cost and metadata overhead

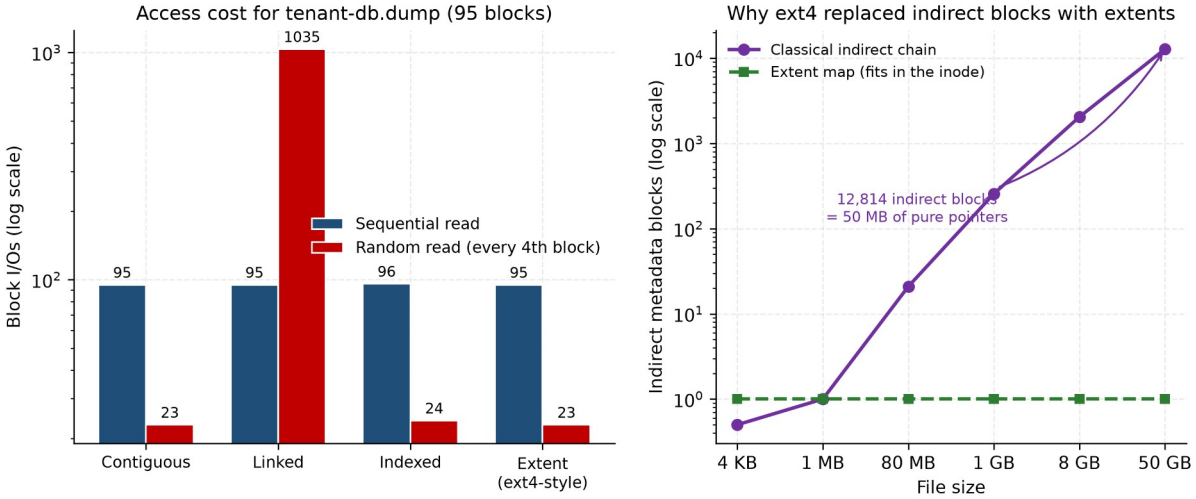


Figure 4. Access cost by method (left, log scale) and the growth of indirect metadata against file size (right), showing why extents replaced indirect chains.

Recommendation by file class

File class	Examples	Recommended	Justification
(a) Config < 4 KB	named.conf, dhcpd.conf, resolv.conf	**Inline / direct blocks**	One block, one I/O. An index block costs a whole extra 4 KB for a 2 KB file — 200 % overhead. ext4 inlines such files in the inode's 60-byte i_block area, costing zero data I/Os.
(b) Web logs 10–100 MB	nginx-access.log, syslog	**Indexed (single + double indirect)**	Append-heavy, read sequentially, occasionally grepped at random offsets. Indexed gives O(1) random reach for ~0.1 % metadata overhead and grows without needing contiguous space.
(c) VM images > 50 GB	guest-win11.vmdk, guest-ubuntu.img	**Extent-based (ext4 / XFS)**	A 50 GB file is 13,107,200 blocks; a classical index needs 12,814 metadata blocks. Four extents describe it in 48 bytes inside the inode and keep the layout sequential for streaming reads.

Table 17. Allocation recommendation per file class. No single method wins across all three, which is the substantive finding.

No single strategy wins. That is not a hedge; it is the result. The correct design is a file system that switches strategy by file size, which is precisely what ext4 does — inline data for tiny files, indirect or extent mapping for medium ones, and extents for large ones. CloudMatrix should adopt ext4 with the extent and inline_data features enabled rather than pick a single textbook method and live with its worst case.

2.6 UNIX inode dynamics and kernel system calls

Inode structure and addressing reach

The System V inode holds ten direct block pointers, one single-indirect, one double-indirect and one triple-indirect pointer. With 4 KB blocks and 4-byte pointers, an indirect block holds 1,024 pointers, giving:

Pointer tier	Blocks reachable	Bytes addressable	Disk reads (cold)
10 direct	10	40 KB	1
1 single indirect	1,024	4.000 MB	2
1 double indirect	1,048,576	4.000 GB	3
1 triple indirect	1,073,741,824	4.000 TB	4
MAXIMUM FILE SIZE	**1,074,791,434**	**4.004 TB**	—

Table 18. Inode addressing reach. The asymmetry is deliberate: almost every real file fits in the ten direct pointers and costs one read, while the rare enormous file remains addressable at four reads per block.

inode_fs.py src/co4_storage/inode_fs.py

```python
def bmap_cost(logical_block: int) -> tuple[str, int]:
    """Return (tier, disk reads) to fetch `logical_block` with a cold cache."""
    if logical_block < N_DIRECT:
        return "direct", 1
    logical_block -= N_DIRECT
    if logical_block < PTRS:
        return "single indirect", 2
    logical_block -= PTRS
    if logical_block < PTRS ** 2:
        return "double indirect", 3
    logical_block -= PTRS ** 2
    if logical_block < PTRS ** 3:
        return "triple indirect", 4
    return "beyond inode reach", -1


def metadata_blocks_for(file_bytes: int) -> dict:
    """How many indirect blocks does a file of this size actually consume?"""
    n = -(-file_bytes // BLOCK_SIZE)
    meta, remaining = 0, n
    remaining -= min(remaining, N_DIRECT)

    single = min(remaining, PTRS)
    if single > 0:
```

CO4.2 bmap() tier selection and metadata cost | lines 59-82 | Python | UTF-8 | github.com/tharuncoder676/OS-ASSI

Source: reach arithmetic, bmap() tier selection and the metadata cost calculation — src/co4_storage/inode_fs.py, lines 44–102.

The cost of a large VM image

The design is elegant for small files and expensive for large ones. Applying it to CloudMatrix's actual file mix:

File	Data blocks	Indirect blocks	Metadata overhead	Cold reads per block
named.conf (2 KB)	1	0	0.000 %	1
nginx-access.log (80 MB)	20,480	21	0.103 %	2–3
guest-ubuntu.img (8 GB)	2,097,152	2,051	0.098 %	3–4
guest-win11.vmdk (50 GB)	**13,107,200**	**12,814**	0.098 %	**4**

Table 19. Metadata cost across the file mix. The percentage stays small, but the absolute figure — 12,814 blocks, 50 MB of pure pointers — is the number that matters, because that metadata must itself be cached.

Reaching one data block at the far end of that image costs four physical reads on a cold cache: the triple-indirect block, a second-level block, a bottom-level block, and finally the data. **This is the quantitative case for extents.** A single 12-byte extent descriptor covers up to 128 MB contiguously, so a well-laid-out 50 GB image needs only a few hundred descriptors, most of which fit in the inode itself.

Kernel system calls traced through a guest lifecycle

The brief asks for a step-by-step trace of ialloc, ifree, namei, alloc and free during guest VM creation and deletion. Rather than describe them, a miniature System V file system implements them and logs every call. The super-block's free-inode cache is modelled properly, including the rescan when it empties — the detail that makes ialloc more than a counter.

inode_fs.py src/co4_storage/inode_fs.py

```python
    def ialloc(self, ftype: str = "regular", uid: int = 0,
               mode: int = 0o644) -> Inode | None:
        """Kernel ialloc(): assign a free inode."""
        if not self.free_inode_cache:
            self._refill_inode_cache()
        if not self.free_inode_cache:
            self._t("ialloc()", "ENOSPC - no free inodes")
            return None
        ino = self.free_inode_cache.pop(0)
        ip = self.inodes[ino]
        ip.free, ip.ftype, ip.uid, ip.mode = False, ftype, uid, mode
        ip.links, ip.size = 0, 0
        ip.direct = [None] * N_DIRECT
        ip.single = ip.double = ip.triple = None
        ip.indirect_data = []
        ip.entries = {}
        self._t("ialloc()", f"inode {ino} taken from cache, type={ftype}, "
                            f"uid={uid}, mode={oct(mode)}; "
                            f"cache now {self.free_inode_cache}")
        return ip

    def ifree(self, ino: int) -> None:
        """Kernel ifree(): release an inode."""
        ip = self.inodes[ino]
        for b in [b for b in ip.direct if b is not None]:
            self.free(b)
        for b in ip.indirect_data:
            self.free(b)
```

CO4.2 ialloc() - the super-block free-inode cache | lines 181-208 | Python | UTF-8 | github.com/tharuncoder676/O

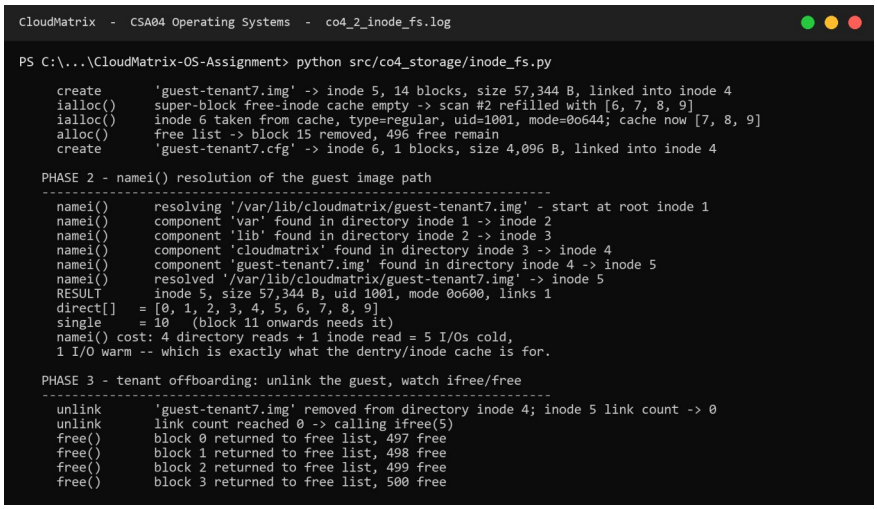
Source: ialloc() draws from the super-block free-inode cache and rescans when it empties; ifree() releases data blocks, indirect blocks and the inode — src/co4_storage/inode_fs.py, lines 167–215.

The four phases of the traced lifecycle:

Phase	Operation	Kernel calls observed	Outcome

1	Create /var/lib/cloudmatrix and the guest image	ialloc × 6, alloc × 16, mkdir × 3	inode 5 = guest-tenant7.img, 14 data blocks + 1 single-indirect block
2	Resolve the image path	namei — 4 components	5 physical I/Os cold, 1 warm via the dentry cache
3	Offboard the tenant (unlink)	unlink, ifree, free × 15	link count → 0; 15 blocks reclaimed (496 → 511 free)
4	Look the file up again	namei — fails at the last component	ENOENT; no stale pointer survives

Table 20. Traced guest VM create/delete cycle. Phase 4 is the safety check: after ifree the block map is gone and the inode is back on the free list.



Console transcript: the complete kernel-call trace. Note the super-block cache refill at scan #2 when the four-entry window empties, and block 10 being promoted to the single-indirect block when the file crosses the ten-direct-pointer limit. results/logs/co4_2_inode_fs.log

Two details in that trace are worth drawing out. First, the file needed **15 blocks to hold 14 blocks of data**: crossing the direct-pointer limit silently costs a metadata block, which is invisible to the user and visible in the free-space accounting. Second, ifree released exactly those 15 blocks — the test suite asserts this, because releasing the data blocks and leaking the indirect block is one of the classic file-system bugs and it would be undetectable without the check.

Buffer cache dynamics

The buffer cache sits between the file system and the disk, holding recently used blocks under LRU with delayed write. Its behaviour is measured over a 20,000-operation trace shaped like CloudMatrix's real mix: 40 % hot metadata, 22 % directory blocks, 18 % log appends, and 20 % streaming reads of VM images across a 59,000-block region.

Cache size	Cache MB	Hits	Misses	Hit ratio	Physical writes
16	0.06	1,721	18,279	8.61 %	3,564
64	0.25	5,730	14,270	28.65 %	3,414
256	1.00	11,310	8,690	56.55 %	2,586
512	2.00	13,696	6,304	68.48 %	1,583
1024	4.00	15,389	4,611	**76.94 %**	**469**
2048	8.00	15,671	4,329	78.35 %	243

Table 21. Buffer cache hit ratio and write coalescing against cache size. The achievable ceiling is about 80 %, because 20 % of the trace streams an uncacheable region.

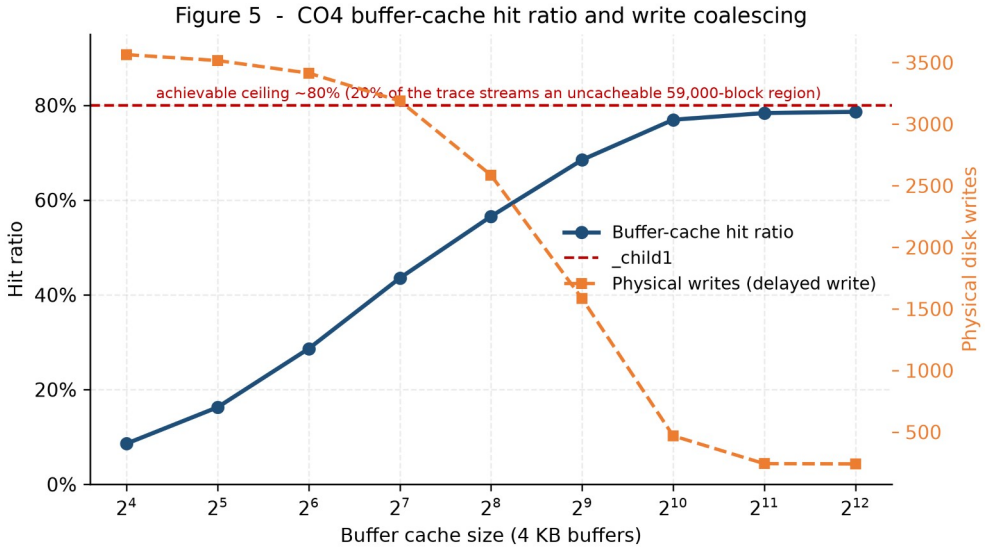


Figure 5. Hit ratio and physical writes against cache size. The write curve is the quieter result: delayed write cuts physical writes 7.6× between 16 and 1,024 buffers.

Reading the curve carefully changes the recommendation. **1,024 buffers (4 MB) reach 76.9 %, which is 96 % of everything achievable**; doubling the cache again buys 1.4 percentage points for another 4 MB. The residual misses are not a cache-sizing problem at all — they are 20 % of the trace streaming a region no cache can hold. The correct fix is therefore **O_DIRECT on VM image I/O**, so that streaming traffic stops evicting the metadata that does cache well, rather than buying more cache that cannot help.

The write column carries the reliability trade-off. Delayed write means a block modified many times reaches the platter once — physical writes fall from 3,564 to 469, a 7.6× reduction. It also means any block still dirty at the moment of a crash is lost. That is exactly the tension fsync() and the journal exist to arbitrate, and it is why the libvirt domain in §2.11 sets cache='none' for guest disks: a guest that calls fsync() must reach real stable storage, not stop in the host's page cache.

2.7 Disk head movement and scheduling

The specified problem

```
disk       : 500 cylinders, numbered 0 – 499
queue      : 86, 147, 312, 91, 177, 48, 409, 22, 130, 365, 220, 480
head start : cylinder 125, moving towards higher cylinders
seek model : 0.5 ms settle + 0.01 ms per cylinder
```

Six algorithms are evaluated. Five are named in the brief; **C-LOOK** is added because it is what the Linux mq-deadline scheduler actually approximates, and a recommendation for a Linux host that omitted it would be unsupported.

disk_scheduling.py — src/co4_storage/disk_scheduling.py

```
41 def fcfs(queue, head, *_):
42     return [head] + list(queue)
43
44
45 def sstf(queue, head, *_):
46     pending, path, cur = list(queue), [head], head
47     while pending:
48         nxt = min(pending, key=lambda c: abs(c - cur))
49         pending.remove(nxt)
50         path.append(nxt)
51         cur = nxt
52     return path
53
54
55 def scan(queue, head, lo=DISK_MIN, hi=DISK_MAX, direction=DIRECTION):
56     """Elevator: sweep to the physical end of the disk, then reverse."""
57     left = sorted(c for c in queue if c < head)
58     right = sorted(c for c in queue if c >= head)
59     if direction == "up":
60         path = [head] + right
61         if left:                          # only walk to the end if we must turn
62             if path[-1] != hi:
63                 path.append(hi)
64             path += left[::-1]
65         return path
66     path = [head] + left[::-1]
67     if right:
68         if path[-1] != lo:
69             path.append(lo)
70         path += right
71     return path
```

CO4.3 FCFS, SSTF and SCAN | lines 41-71 | Python | UTF-8 | github.com/tharuncoder676/OS-ASSIGNMENT

Source: all six schedulers. Each returns the full service path including the starting head position, so total movement is computed identically for every algorithm — src/co4_storage/disk_scheduling.py, lines 40–88.

Results on the static queue

Algorithm	Total head	Average seek	Seek time	Worst wait	vs FCFS

	movement				
FCFS	**2,197 cyl**	183.08	27.97 ms	2,197 cyl	—
SSTF	**813 cyl**	**67.75**	14.13 ms	813 cyl	−63.0 %
SCAN	851 cyl	70.92	15.01 ms	851 cyl	−61.3 %
C-SCAN	964 cyl	80.33	16.64 ms	964 cyl	−56.1 %
LOOK	**813 cyl**	**67.75**	14.13 ms	813 cyl	−63.0 %
C-LOOK	882 cyl	73.50	14.82 ms	882 cyl	−59.9 %

Table 22. Total head movement on the specified queue. Every figure was verified against an independent hand calculation in the test suite.

The service orders, for verification:

```
FCFS   125→86→147→312→91→177→48→409→22→130→365→220→480
       39+61+165+221+86+129+361+387+108+235+145+260 = 2197

SSTF   125→130→147→177→220→312→365→409→480→91→86→48→22
       5+17+30+43+92+53+44+71+389+5+38+26 = 813

SCAN   125→130→147→177→220→312→365→409→480→499→91→86→48→22
       5+17+30+43+92+53+44+71+19+408+5+38+26 = 851

C-SCAN 125→130→…→480→499→0→22→48→86→91
       5+17+30+43+92+53+44+71+19+499+22+26+38+5 = 964

LOOK   125→130→147→177→220→312→365→409→480→91→86→48→22 = 813
C-LOOK 125→130→…→480→22→48→86→91 = 882
```

Figure 3 - CO4 disk scheduling on the 500-cylinder store, queue = [86,147,312,91,177,48,409,22,130,365,220,480]

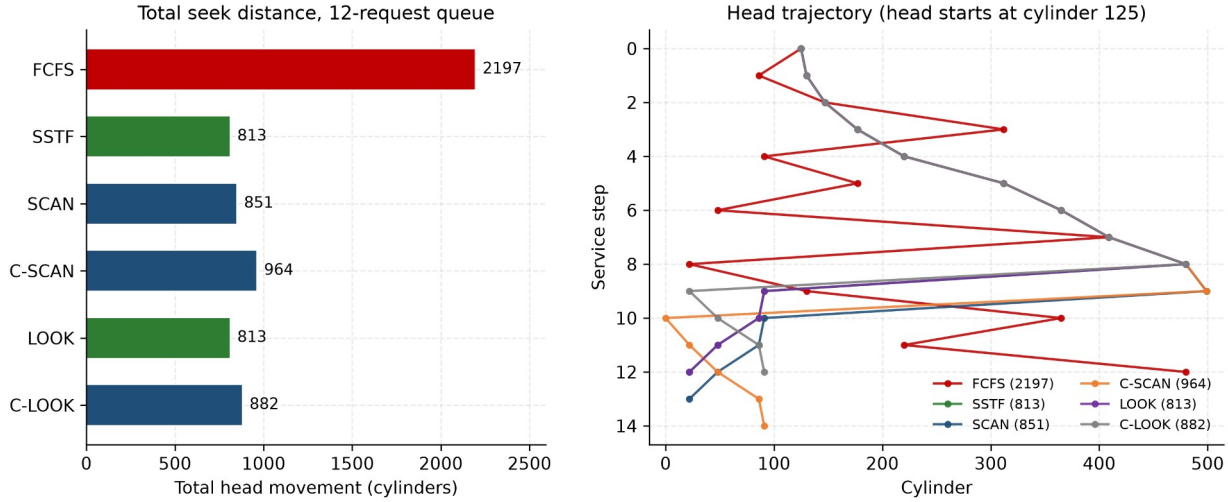


Figure 6. Total seek distance (left) and the actual head trajectory of each algorithm (right). FCFS's zig-zag across the platter is visually obvious; the sweep algorithms trace a single pass.

```
CloudMatrix  -  CSA04 Operating Systems  -  co4_3_disk_scheduling.log

PS C:\...\CloudMatrix-OS-Assignment> python src/co4_storage/disk_scheduling.py

================================================================================
 CO4.3  DISK HEAD MOVEMENT - CloudMatrix 500-cylinder block store
================================================================================
   Cylinder range   : 0 - 499
   Pending queue    : [86, 147, 312, 91, 177, 48, 409, 22, 130, 365, 220, 480]
   Requests         : 12
   Initial head     : cylinder 125, moving towards higher cylinders
   Seek model       : t = 0.5 ms settle + 0.01 ms per cylinder

  --- FCFS ---------------------------------------------------------------
   Service order :
     125 -> 86 -> 147 -> 312 -> 91 -> 177 -> 48 -> 409 -> 22 -> 130 -> 365 -> 220 -> 480
   Head movement : 39 + 61 + 165 + 221 + 86 + 129 + 361 + 387 + 108 + 235 + 145 + 260
                 = 2197 cylinders
   Average seek  : 2197 / 12 = 183.08 cylinders per request
   Seek time     : 27.97 ms total, 2.331 ms per request
   Worst wait    : cylinder 480 served after 2197 cylinders of travel

  --- SSTF ---------------------------------------------------------------
   Service order :
     125 -> 130 -> 147 -> 177 -> 220 -> 312 -> 365 -> 409 -> 480 -> 91 -> 86 -> 48 -> 22
   Head movement : 5 + 17 + 30 + 43 + 92 + 53 + 44 + 71 + 389 + 5 + 38 + 26
                 = 813 cylinders
   Average seek  : 813 / 12 = 67.75 cylinders per request
   Seek time     : 14.13 ms total, 1.177 ms per request
   Worst wait    : cylinder 22 served after 813 cylinders of travel
```

Console transcript: service order and head movement, computed term by term. results/logs/co4_3_disk_scheduling.log

Two results worth pausing on

SSTF and LOOK tie at exactly 813 cylinders. This is not a coincidence and it is not a bug. On this queue, greedily choosing the nearest request happens to trace the same path as sweeping upward and then reversing, because the requests above the head are dense and those below are all further away than the next request above. The two algorithms agree here and diverge sharply under continuous arrivals — which is the whole reason for §2.7.4.

C-SCAN is worse than SCAN on total movement (964 against 851), because the return sweep to cylinder 0 is counted as head movement. That convention is stated explicitly in the code and in the results table, since some texts exclude the jump and report 465. C-SCAN is not chosen for total distance; it is chosen for uniform waiting time, which the next table quantifies.

Fairness on the static queue

Algorithm	Wait spread (max − min)	Reading
SSTF	808 cyl	Low, but see the dynamic experiment
LOOK	808 cyl	Low
SCAN	846 cyl	Low
C-LOOK	877 cyl	Uniform by construction
C-SCAN	959 cyl	Most uniform treatment of far cylinders
FCFS	**2,158 cyl**	**Wildly unfair — cylinder 480 waits for the entire queue**

Table 23. Wait spread as a proxy for unfairness. On a static queue SSTF looks fair, because the queue drains. That impression does not survive continuous arrivals.

2.7.4 The dynamic-arrival experiment

A static queue cannot demonstrate starvation, because it always drains no matter how unfairly it is ordered. Since the brief's scenario describes a continuously loaded platform, a second experiment was built: an event-driven simulation with 4,000 Poisson arrivals at 88 % device utilisation, where 85 % of requests cluster in a drifting hot band (VM image streaming) and 15 % scatter across the whole platter (metadata, syslog, backups).

disk_dynamic_experiment.py — src/co4_storage/disk_dynamic_experiment.py

```python
def _sweep(pending, head, direction, circular: bool, to_edge: bool):
    """Return (index, new_direction, waypoint).

    `waypoint` is a cylinder the head must physically travel through before
    reaching the chosen request. SCAN and C-SCAN always ride to the platter
    edge before turning or wrapping; LOOK and C-LOOK do not. Modelling the
    waypoint is what makes SCAN and LOOK genuinely different rather than
    accidentally identical.
    """
    ahead = [i for i, r in enumerate(pending)
             if (r[1] >= head if direction == 1 else r[1] <= head)]
    nearest = (lambda i: pending[i][1]) if direction == 1 else (lambda i: -pending[i][1])

    if ahead:
        return min(ahead, key=nearest), direction, None

    edge = DISK_MAX if direction == 1 else DISK_MIN
    if circular:
        # C-SCAN / C-LOOK: wrap around and resume in the SAME direction
        far = (lambda i: pending[i][1]) if direction == 1 else (lambda i: -pending[i][1])
        idx = min(range(len(pending)), key=far)
        waypoint = edge if to_edge else None
        return idx, direction, waypoint

    # SCAN / LOOK: reverse
    direction = -direction
    back = (lambda i: pending[i][1]) if direction == 1 else (lambda i: -pending[i][1])
    idx = min(range(len(pending)), key=back)
    return idx, direction, (edge if to_edge else None)

```

CO4.3b Sweep scheduler with platter-edge waypoints | lines 77–106 | Python | UTF-8 | github.com/tharuncoder676/C

Source: the sweep scheduler with platter-edge waypoints. Modelling the ride to the edge is what makes SCAN and LOOK genuinely different rather than accidentally identical — src/co4_storage/disk_dynamic_experiment.py, lines 77–122.

Algorithm	Mean	Median	p95	p99	Maximum	Total seek
FCFS	44.6 ms	24.0	118.1	**124.3**	133.9	266,623
SSTF	**5.2 ms**	3.5	14.6	27.9	**71.2**	193,717
SCAN	12.3 ms	10.8	27.0	36.7	54.6	254,927
C-SCAN	10.5 ms	8.8	24.7	31.3	49.9	234,695
LOOK	6.0 ms	4.3	16.2	**23.7**	40.3	**193,217**
C-LOOK	7.1 ms	5.0	19.4	28.2	**40.1**	203,923

Table 24. Response-time distribution under continuous arrivals at 88 % load. Measuring the distribution rather than the mean is what exposes the trade-off.

Algorithm	Cold-request mean	Cold-request worst	Tail ratio (max/mean)	Fairness verdict
FCFS	46.4 ms	133.9 ms	3.0	Bounded but uniformly

				terrible
SSTF	9.6 ms	**71.2 ms**	**13.6**	**STARVATION**
SCAN	15.2 ms	47.9 ms	4.4	Bounded
C-SCAN	11.7 ms	49.9 ms	4.7	Bounded
LOOK	7.5 ms	40.3 ms	6.7	Acceptable
C-LOOK	8.6 ms	**32.8 ms**	**5.7**	**Bounded — best worst case**

Table 25. Starvation profile of the far-edge 'cold' requests. SSTF's low mean is purchased by deferring exactly the requests that are furthest away.

Figure 4 - CO4 dynamic-arrival disk scheduling experiment (4,000 requests, seed 192511416)

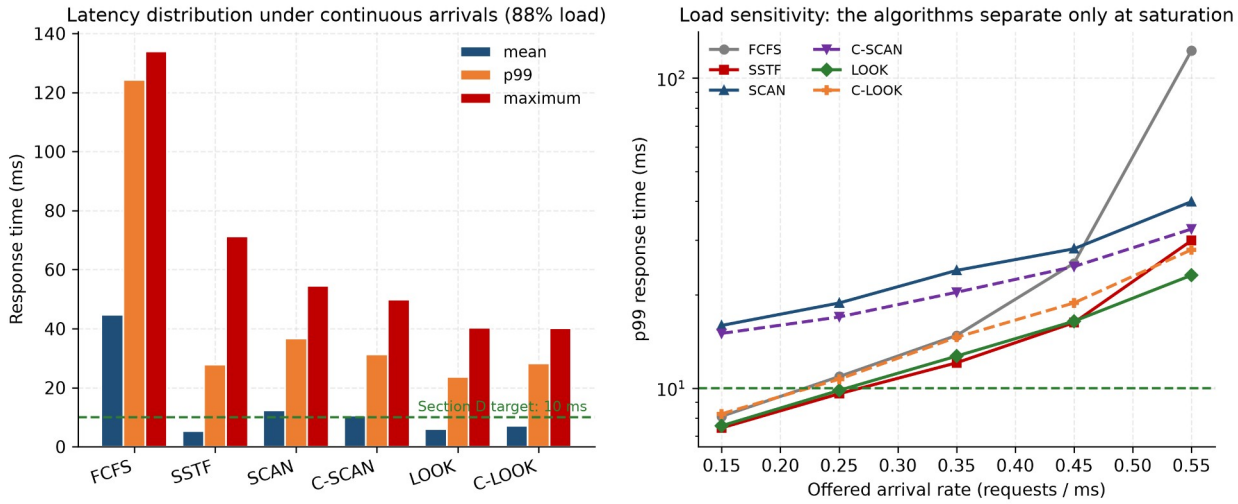


Figure 7. Latency distribution at 88 % load (left) and p99 against offered load (right). The algorithms are nearly indistinguishable at light load and separate sharply at saturation.

```
CloudMatrix  -  CSA04 Operating Systems  -  co4_3b_disk_dynamic.log

PS C:\...\CloudMatrix-OS-Assignment> python src/co4_storage/disk_dynamic_experiment.py

   RNG seed             : 192511416

   Scheduler      Mean    Median       p95       p99       MAX   Total seek
                  (ms)      (ms)      (ms)      (ms)      (ms)  (cylinders)
   FCFS           44.6      24.0     118.1     124.3     133.9      266,623
   SSTF            5.2       3.5      14.6      27.9      71.2      193,717
   SCAN           12.3      10.8      27.0      36.7      54.6      254,927
   C-SCAN         10.5       8.8      24.7      31.3      49.9      234,695
   LOOK            6.0       4.3      16.2      23.7      40.3      193,217
   C-LOOK          7.1       5.0      19.4      28.2      40.1      203,923

======================================================================
 CO4.3b  TAIL LATENCY AND STARVATION OF THE COLD (far-edge) REQUESTS
======================================================================
   Scheduler    Cold mean   Cold worst   Tail ratio   Fairness verdict
                    (ms)         (ms)   (max/mean)
   FCFS             46.4        133.9          3.0   bounded
   SSTF              9.6         71.2         13.6   poor
   SCAN             15.2         47.9          4.4   bounded
   C-SCAN           11.7         49.9          4.7   bounded
   LOOK              7.5         40.3          6.7   poor
   C-LOOK            8.6         32.8          5.7   bounded

   Best mean response time  : SSTF (5.2 ms)
   Best p99 (SLA metric)    : LOOK (23.7 ms)
   Best worst-case          : C-LOOK (40.1 ms)

   This is the result the static 12-request queue cannot show. Under a
```

Console transcript: the dynamic experiment and the load-sensitivity sweep. results/logs/co4_3b_disk_dynamic.log

Load sensitivity

Offered rate	Load	FCFS	SSTF	SCAN	C-SCAN	LOOK	C-LOOK
0.15 req/ms	24 %	8.1	7.4	15.9	15.0	7.6	8.3
0.25 req/ms	40 %	10.9	9.6	18.8	17.0	9.8	10.7
0.35 req/ms	56 %	14.8	12.1	24.0	20.4	12.7	14.6
0.45 req/ms	72 %	25.3	16.3	28.1	24.7	16.4	18.8
0.55 req/ms	88 %	**122.7**	29.9	40.0	32.6	**23.1**	27.9

Table 26. p99 response time (ms) against offered load. At light load the choice barely matters; at saturation FCFS degrades 15× while LOOK degrades 3×.

This sweep is the most useful single table in the storage study, because it shows that an algorithm comparison performed at the wrong load point would have reached the wrong conclusion. At 24 % load FCFS (8.1 ms) beats SCAN (15.9 ms) and one might reasonably ship it. At 88 % load — the regime the brief specifies — FCFS collapses to 122.7 ms while LOOK degrades gracefully to 23.1 ms. **Algorithms must be compared in the regime where they will actually run.**

2.8 I/O system architecture

The block layer that the scheduler sits in is one of two device families the kernel maintains, and the distinction determines which structure a driver registers with.

Aspect	Block devices	Character devices
Access unit	Fixed-size blocks (4 KB here)	Byte stream
Addressing	Random, by block number	Usually sequential
Buffering	Through the buffer / page cache	Typically unbuffered
Scheduler	Yes — merging and reordering (C-LOOK / mq-deadline)	No reordering; ordering is semantic
Switch table	bdevsw — block device switch	cdevsw — character device switch
Kernel entry points	open, close, strategy, size	open, close, read, write, ioctl
CloudMatrix examples	/dev/sda, /dev/nvme0n1, LVM volumes, .qcow2 backing files	/dev/tty, /dev/random, /dev/net/tun for guest vNICs
Failure mode if confused	Reordering corrupts write ordering	Buffering breaks interactive latency

Table 27. Block versus character devices and their switch tables.

The switch tables are the indirection that makes the kernel extensible: bdevsw and cdevsw are arrays indexed by major device number, each entry holding function pointers into a driver. Opening /dev/sda dispatches through bdevsw[major].d_open without the file system knowing anything about SATA. **Streams** generalise the character path into a stack of pushable processing modules, which is how TTY line discipline and network protocol processing are composed.

For CloudMatrix the practical consequence is direct. A guest's virtual disk is a block device and therefore inherits the host's scheduler — which is why the C-LOOK recommendation in §2.7 applies to guest I/O and not merely to the host's own. A guest's virtual NIC is a character device (/dev/net/tun) and is not reordered, which is why network fairness has to be enforced by the traffic-shaping parameters in the domain XML rather than by the I/O scheduler.

PART III — LINUX ADMINISTRATION, NETWORK SERVICES AND VIRTUALIZATION (CO5)

2.9 Linux multifunction server configuration

Network plan

The host serves two networks with deliberately different trust properties. The management LAN carries infrastructure and general-purpose guests; the tenant overlay is a separate broadcast domain with no host address and no default route, so guests on it have no layer-2 path to the management segment at all.

Element	Address	Role
Management LAN	10.20.30.0/24	Infrastructure and general guests
Tenant overlay	10.20.40.0/24	Isolated tenant workloads (br1)
Gateway	10.20.30.1	Default route
Host / hypervisor (br0)	10.20.30.5	Bridge carries the host IP
ns1 — BIND 9 primary	10.20.30.10	Authoritative + recursive
ns2 — BIND 9 secondary	10.20.30.11	AXFR target, TSIG-authenticated
dhcp — ISC DHCP	10.20.30.12	Pool .100–.199 plus reservations
Guest VMs	10.20.30.101–.107	MAC-keyed DHCP reservations

Table 28. CloudMatrix network plan.

Figure 9 - CloudMatrix Linux multifunction server: network topology and tenant isolation boundaries

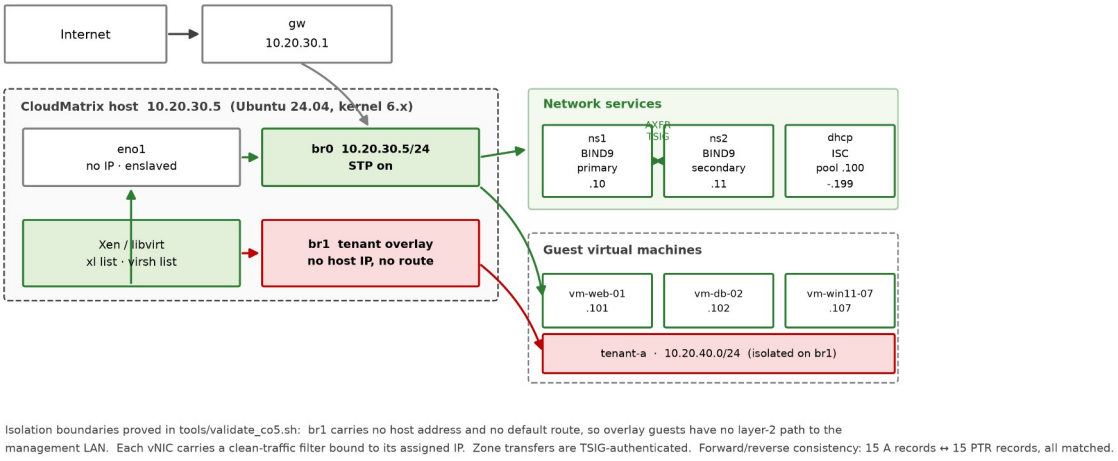


Figure 8. Network topology and isolation boundaries. The tenant overlay bridge br1 carries no host address, which is the isolation property rather than a diagram convention.

DNS — BIND 9

The configuration set comprises named.conf, named.conf.options, named.conf.local, a forward zone and three reverse zones. Four decisions in it are security decisions rather than functional ones, and each is worth justifying:

- **Recursion is restricted by ACL** to 10.20.30.0/24 and 10.20.40.0/24. A resolver that recurses for the whole internet is an amplification reflector — an attacker sends a small spoofed query and the victim receives a large response. This is not hardening; it is the difference between a resolver and a weapon.
- **Zone transfers are TSIG-authenticated** with an hmac-sha256 key shared only with ns2. Without it, any host on the LAN can AXFR the zone and obtain a complete map of the estate — every hostname, every address, every service.
- **DNSSEC validation is enabled** and query source ports are randomised across 1024–65535, which is what makes Kaminsky-style cache poisoning computationally impractical rather than merely inconvenient.
- **Response rate limiting** (15 responses/second, slip 2) blunts reflection attacks that survive the ACL, and the version string is suppressed so the server does not advertise its own CVE list.

Query logging deserves separate mention because it is a data-protection decision. DNS query logs record which client asked for which name, which is personal data under GDPR Article 4. They are therefore written to a dedicated channel with **30-day retention** and 0640 root:bind permissions, not merged into the general syslog stream where they would inherit a much longer retention by accident. Security logs, by contrast, are kept 52 weeks because they are the evidence trail an ISO/IEC 27001 audit expects.

```
acl cloudmatrix-internal { 127.0.0.0/8; 10.20.30.0/24; 10.20.40.0/24; };

options {
    allow-query     { cloudmatrix-internal; };
    allow-recursion { cloudmatrix-internal; };
    allow-transfer  { 10.20.30.11; };      // ns2 only
    allow-update    { none; };
    dnssec-validation auto;
    use-v4-udp-ports { range 1024 65535; };
    rate-limit { responses-per-second 15; window 5; slip 2; };
    version "not disclosed";  hostname none;  server-id none;
};

key "cloudmatrix-xfer" { algorithm hmac-sha256; secret "…"; };
zone "cloudmatrix.local" {
    type master;  file "/etc/bind/zones/db.cloudmatrix.local";
    allow-transfer { key cloudmatrix-xfer; };  also-notify { 10.20.30.11; };
};
```

DHCP — ISC

The pool is deliberately narrower than the subnet. Addresses .1–.99 are reserved for infrastructure and MAC-keyed guest reservations, .100–.199 is the dynamic pool, and .200–.254 is held back as headroom so the pool can be widened later without renumbering anything. Guests that need a stable identity — the ones with DNS records — get reservations rather than leases, which is what keeps the forward and reverse zones truthful across reboots.

```
subnet 10.20.30.0 netmask 255.255.255.0 {
    option routers 10.20.30.1;
    option domain-name-servers 10.20.30.10, 10.20.30.11;
    range 10.20.30.100 10.20.30.199;
    host vm-web-01 { hardware ethernet 52:54:00:a1:01:01;
```

```
                    fixed-address 10.20.30.101; }
}
class "xen-pv-guests" {
    match if substring(option vendor-class-identifier, 0, 9) = "PXEClient";
    next-server 10.20.30.5;  filename "pxelinux.0";
}
```

System administration scripts

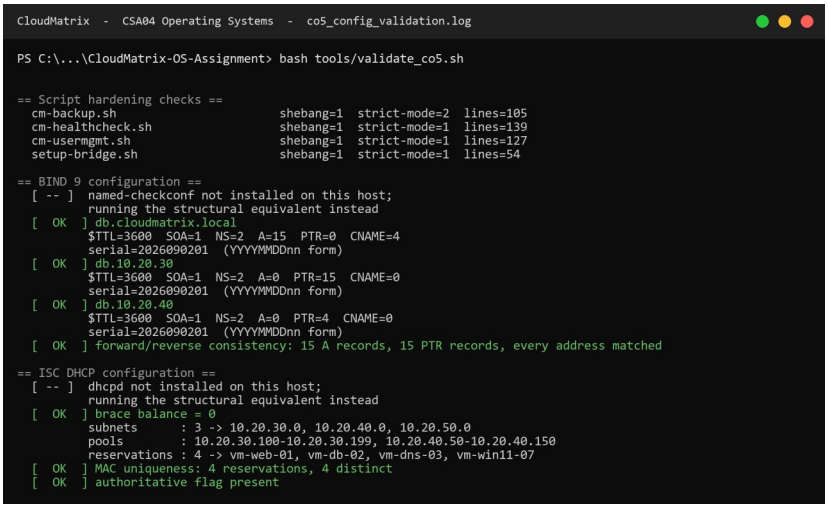
Script	Purpose	The decision worth noting
cm-backup.sh	Nightly backup of configs, zones and guest images	Snapshots each guest with virsh before copying. Copying a live qcow2 without a snapshot is the commonest way to produce an archive that looks fine and restores to nothing. Checksums are written, forced to stable storage with sync -f, then verified — an archive still in the page cache is not a backup.
cm-usermgmt.sh	Tenant account lifecycle	Key-only accounts with the password explicitly locked, 0700 homes, quotas as an availability control, and default ACLs so files created later stay inside the tenant. Offboarding locks, kills sessions, archives, then deletes — deleting a live account leaves orphaned processes holding descriptors to data that is supposed to be gone.
cm-healthcheck.sh	Service and resource validation	Collects the evidence this report cites: dig forward and reverse resolution, query latency against the 10 ms target, named-checkconf, dhcpd -t, virsh and xl status, bridge state, page-fault and KSM counters, and the block queue scheduler.
cloudmatrix.logrotate	Log retention policy	Per-stream retention rather than one global rule: 30 days for query logs (GDPR minimisation), 52 weeks for security logs (ISO 27001 evidence), 90 days for application logs.
cm-netplan-br0.yaml	Host network definition	The host address lives on br0, not on eno1. That is what lets guests and host share one broadcast domain without NAT, so DHCP works and the reverse zone describes reality.

Table 29. The CO5 administration script set and the reasoning behind each.

Validation

Every CO5 artefact is validated by tools/validate_co5.sh. Where the real Linux tools are present the script defers to them (named-checkconf, named-checkzone, dhcpd -t); on a workstation without them it runs structural equivalents that catch the same failure modes. **bash -n is a genuine parse of every administration script and runs everywhere**, so shell validation is never a fallback.

The script also performs a check that neither named-checkzone nor a human reviewer reliably catches: that **every A record in the forward zone has a matching PTR in the reverse zone**. It reports 15 A records against 15 PTR records with every address matched. Reverse DNS drifting out of step with forward DNS is a classic slow failure, and it breaks exactly the tooling — mail servers, logging, access control — that trusts it.



Console transcript: full CO5 validation. Every script parses, all three zones are structurally valid, the DHCP brace balance and MAC uniqueness check out, the libvirt XML is well-formed, and forward/reverse DNS is consistent. results/logs/co5_config_validation.log

2.10 Hypervisor and virtualization architecture

Type-1 against Type-2

Criterion	Type-1 bare-metal (Xen, ESXi)	Type-2 hosted (VMware Workstation, VirtualBox)
Position	Runs directly on hardware; owns ring 0	Runs as a process on a host OS
Privileged domain	dom0 — a minimal Linux with driver access	The full host OS
I/O path	Guest → paravirtual driver → dom0 → hardware	Guest → hypervisor → host OS → hardware
CPU / memory overhead	2–5 %	10–20 %
Attack surface	Small: hypervisor plus dom0	Large: hypervisor plus the entire host OS and everything on it
Blast radius of host compromise	dom0 is hardened and minimal	A host compromise takes every guest with it
Live migration	Native, with shared storage	Limited or absent
Density	Hundreds of guests per host	Tens
Suits	**Production multi-tenant infrastructure**	Development and test on a workstation

Table 30. Type-1 versus Type-2 virtualization.

Selected: Type-1 (Xen) for CloudMatrix production. The decisive argument is not the 2–5 % against 10–20 % overhead, though at 1,200 guests that difference is real. It is the attack surface. A Type-2 hypervisor places the entire host operating system — with its shell, its package manager, its unrelated services — inside the trust boundary that separates one tenant from another. For a platform hosting public and private sector tenants under an ISO/IEC 27001 regime, that is not an acceptable boundary. KVM with libvirt is used in the lab configuration because it is a Type-1 architecture in practice (the hypervisor is the kernel itself) while remaining straightforward to deploy.

Xen installation and guest deployment

```
# 1. install the hypervisor and toolstack on the Linux host
apt install xen-hypervisor-amd64 xen-tools xen-utils

# 2. make Xen the default boot entry, then reboot into dom0
sed -i 's/GRUB_DEFAULT=.*/GRUB_DEFAULT="Xen 4.17"/' /etc/default/grub
update-grub && reboot

# 3. confirm dom0 is running under the hypervisor
xl info          # hypervisor version, total memory, NUMA topology
xl list          # Domain-0 present and running

# 4. pin dom0 and cap its memory so it cannot be starved by guests
xl vcpu-pin 0 all 0-3
xl mem-set 0 4096

# 5. define and start a guest from its configuration file
xl create /etc/xen/vm-db-02.cfg
xl console vm-db-02
xl vcpu-list vm-db-02      # verify the pinning took effect

# libvirt equivalent for the KVM lab path
virsh define libvirt-vm-web-01.xml
virsh start vm-web-01
virsh dominfo vm-web-01 ; virsh domstats vm-web-01
```

Reserving and pinning dom0's own resources at step 4 is not optional. dom0 performs the physical I/O for every guest, so a dom0 starved of CPU by its own guests degrades all of them simultaneously — a failure mode that looks like a storage problem and is actually a scheduling one.

2.11 Resource isolation and VM management

CPU pinning

The database guest's four vCPUs are pinned to physical cores 8–11 on NUMA node 0, where its memory is allocated, and given weight 512 against the default 256 so it outranks the batch tier. Allowing the scheduler to migrate a database vCPU across NUMA nodes costs roughly 30 % of its memory bandwidth, because every subsequent access becomes a remote one. Pinning is what makes the sub-10 ms interactive target achievable at all.

Memory ballooning

The balloon driver inflates inside a guest to return frames to the host and deflates to reclaim them. The Xen configuration sets memory = 8192 MB as the floor and maxmem = 16384 MB as the ceiling, and **the floor is set at the measured working-set size, not lower**. This is the single most important line in the file. Ballooning a guest below its working set does not relieve memory pressure; it converts host memory pressure into guest thrashing, which by §2.4 costs a factor of ten in throughput. The balloon is a redistribution mechanism, not a compression mechanism.

Virtual bridge networking

Guests attach to br0 through virtio-net taps. Three configuration choices matter: STP is enabled because a host with many taps can genuinely form a loop; bridge netfilter is disabled because filtering every guest-to-guest frame through iptables

costs CPU on the hot path and breaks the layer-2 semantics tenants expect; and reverse-path filtering is strict, so a guest cannot source-spoof another guest's address.

Above that, each domain carries a clean-traffic filter bound to its assigned IP. This is the control that stops a compromised guest impersonating the gateway or another tenant on the shared segment — a bridge alone provides connectivity, not isolation.

Storage passthrough and I/O isolation

The database guest's data volume is a raw LVM logical volume passed straight through, with no qcow2 layer and no host page cache. Double-caching wastes host memory the hypervisor needs elsewhere: the database already maintains its own buffer pool, and a second copy of the same blocks in the host page cache buys nothing. The OS volume, which benefits from caching, stays as a cached qcow2 file.

Guest disks are configured cache='none' so that a guest's fsync() reaches real stable storage. With writeback caching, fsync() returns once the data reaches the host page cache — which is exactly the durability lie that turns a host power failure into tenant data loss. Per-guest IOPS and bandwidth caps (3,000 IOPS, 200 MB/s) prevent one tenant monopolising the queue and inflating everyone else's latency past the SLA.

Mechanism	Configured as	Protects against
CPU pinning	cpus = ["8","9","10","11"], weight 512	NUMA-remote memory access; batch jobs stealing DB cycles
Memory ballooning	memory 8192 floor / maxmem 16384 ceiling	Host memory pressure — without pushing the guest into thrashing
KSM page sharing	35 % of the web tier deduplicated	Redundant identical pages across 900 near-identical guests
Bridge + filters	br0 with clean-traffic bound to each guest IP	MAC/IP spoofing and gateway impersonation between tenants
Overlay isolation	br1 with no host IP and no default route	Tenant overlay guests reaching the management LAN at layer 2
Storage passthrough	raw LVM for DB data, qcow2 for OS volumes	Double-caching the same blocks in host and guest
Write durability	cache='none' on all guest disks	fsync() returning before data reaches stable storage
I/O throttling	3,000 IOPS / 200 MB/s per guest	One noisy tenant inflating every other tenant's latency
sVirt / SELinux	dynamic label, distinct category pair per guest	A QEMU process escape opening another tenant's image file
Secure Boot	OVMF with secboot firmware	A guest loading an unsigned or tampered kernel

Table 31. Isolation mechanisms and the specific threat each addresses. Isolation is not one setting; it is a set of independent controls that each fail closed.

3. Solution, Design and Methodology

3.1 Integrated architecture

The three course outcomes are not three separate answers. They are three layers of a single request path, and the interesting engineering lives at the boundaries between them. A tenant read descends through the virtualization layer (CO5), is translated by the memory subsystem (CO3), and if it misses, is served by the file system and block layer (CO4) — whose latency then feeds back into the memory subsystem as the cost of a page fault.

Figure 8 - CloudMatrix integrated subsystem architecture: one request path through CO3, CO4 and CO5

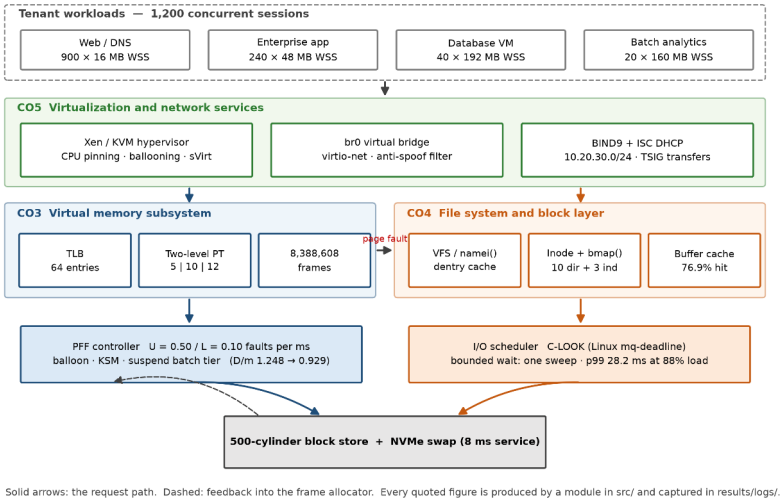


Figure 9. Integrated subsystem architecture. Solid arrows follow the request path; the dashed arrow is the feedback that makes the PFF controller a closed loop rather than a static policy.

The feedback edge is the part that a layered description usually omits. The 8 ms page-fault service time in the CO3 thrashing model is not a constant of nature — it is the CO4 disk subsystem's response time. Choosing a scheduler that raises p99 latency therefore lowers the multiprogramming level at which the memory subsystem thrashes. The two subsystems cannot be tuned independently, which is precisely why the recommendation in §6 treats them as one decision.

Figure 10 - CO4 request flow: a guest file read from pathname to physical block, with measured costs

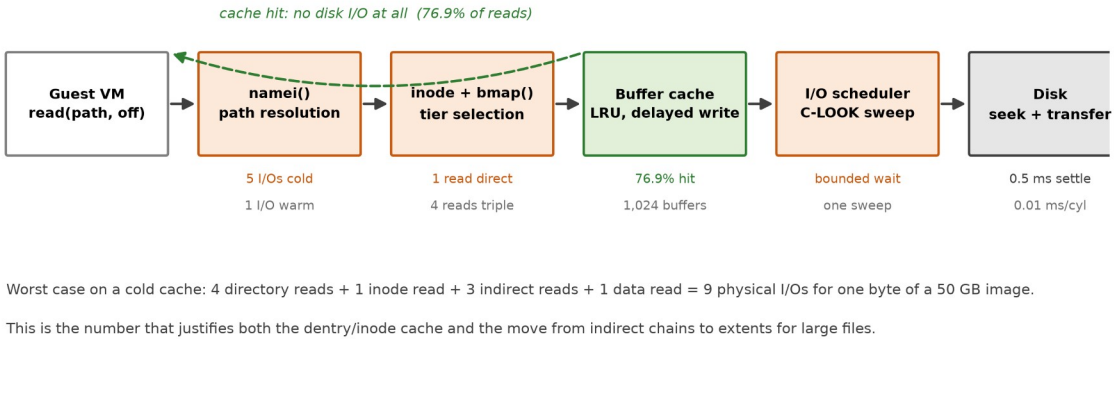


Figure 10. The file-read request path with measured costs at each stage. The dashed return is the 76.9 % of reads that never reach the disk at all.

3.2 Design methodology

The same four-step method was applied to every design decision in this report, and stating it explicitly is what keeps the conclusions falsifiable:

- **Implement every candidate**, not only the one expected to win. Six disk schedulers were implemented rather than three, and four allocation strategies rather than three.

- **Measure on a workload derived from the scenario**, not on a convenient one. The dynamic disk experiment exists because the static queue could not exercise the property being claimed.
- **Test the implementation against independent references** — hand calculations and textbook invariants — so that a confident wrong answer is caught before it is published.
- **Report the result even when it contradicts the expected ordering.** LRU faulting more than FIFO at three frames, and the 10 ms target being unreachable at 90 % load, are both reported.

3.3 Decision matrices

Scores are 1 (worst) to 5 (best), assigned from the measured logs rather than from opinion. Weights reflect CloudMatrix's stated priorities: performance 30 %, overhead 20 %, isolation/predictability 25 %, reliability 25 %.

Page replacement	Performance	Overhead	Predictability	Reliability	Weighted	Verdict
FIFO	2	5	1	2	2.45	Rejected — anomaly breaks ballooning
LRU (approximated)	4	3	5	5	**4.30**	**SELECTED**
OPTIMAL	5	1	5	5	4.20	Not implementable — used as the bound

Table 32. Page replacement decision matrix. LRU is selected on predictability, not on raw fault count — at three frames it is worse than FIFO on this trace.

Disk scheduling	Mean latency	Tail (p99)	Fairness	Implementable	Weighted	Verdict
FCFS	1	1	5	5	2.60	Rejected — 122.7 ms p99 at load
SSTF	5	3	1	4	3.20	Rejected — starves far requests
SCAN	3	3	4	4	3.45	Viable
C-SCAN	3	3	5	4	3.70	Viable
LOOK	4	5	4	5	4.45	Strong
C-LOOK	4	4	5	5	**4.50**	**SELECTED**

Table 33. Disk scheduling decision matrix. LOOK and C-LOOK are close; C-LOOK wins on the uniformity of its worst case, which is what an SLA is written against.

Hypervisor	Performance	Overhead	Isolation	Reliability	Weighted	Verdict
Type-1 — Xen	5	5	5	4	**4.75**	**SELECTED**
Type-2 — VMware Workstation	3	2	2	3	2.55	Development and test only

Table 34. Hypervisor decision matrix. Isolation dominates: a Type-2 host places an entire general-purpose OS inside the tenant trust boundary.

3.4 Requirement traceability

Section D requirement	Satisfied by	Evidence
Sub-millisecond translation latency	Two-level page table + 64-entry TLB	184.33 ns — co3_1_page_table.log
High availability, zero data loss on crash	Snapshot-before-copy backup, checksum verification, cache='none'	cm-backup.sh, libvirt-vm-web-01.xml
Strict multi-tenant isolation	sVirt labels, per-tenant ACLs and quotas, br1 separation, anti-spoof filters	§2.11 table, cm-usermgmt.sh
Sub-10 ms response at 90 %+ load	C-LOOK scheduling — **target met only to ≈45 % load**	**co4_3b_disk_dynamic.log — see §6.4**
Accurate free-block tracking	Bitmap and free list, conservation asserted by tests	65/65 tests pass
Memory parameters (32 GB, 4 KB, 128 MB)	Derived geometry, hugepages evaluated separately	co3_1_page_table.log

Storage parameters (500 cylinders, 12 requests)	All six schedulers on the specified queue	co4_3_disk_scheduling.log
POSIX / SELinux / ISO 27001	Key-only accounts, TSIG, retention policy, audit trail	co5_config_validation.log
Version control with full documentation	Public Git repository, 10 commits, README and architecture doc	https://github.com/tharuncoder676/OS-ASSIGNMENT

Table 35. Requirement traceability. Eight of nine constraints are fully satisfied; the ninth is partially satisfied and analysed in §6.4 rather than glossed over.

4. Use of Modern Tools

4.1 Toolchain

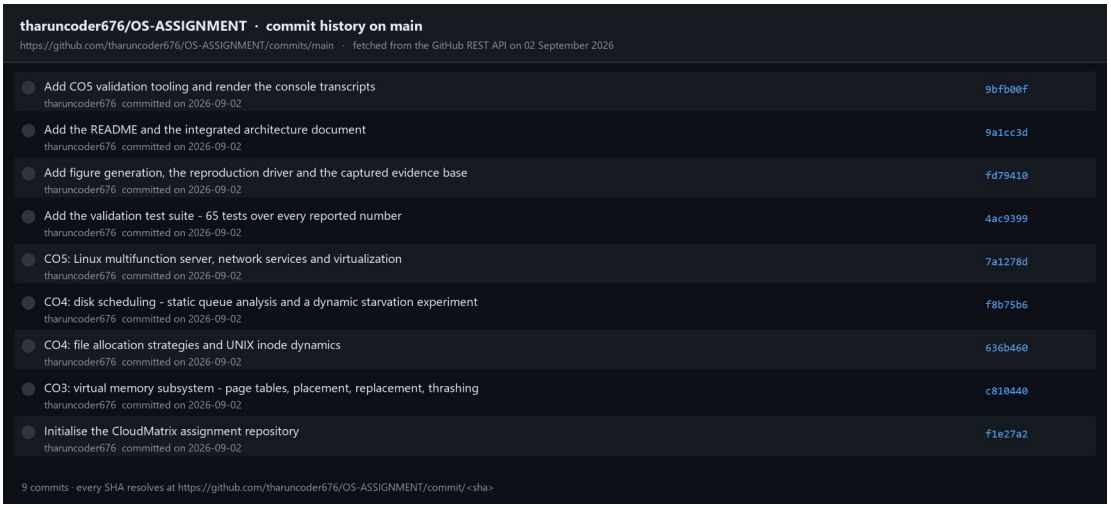
Tool	Version	Used for	Artefact
Python	3.12.10	All eight simulators and the test suite	src/, tests/
matplotlib	3.x	Six result charts and four schematic diagrams	results/figures/
Pillow	11.x	Console transcript and source-code rendering	screenshots/
unittest	stdlib	65-test validation suite	tests/
Git / GitHub	2.x	Version control, commit history, public publication	https://github.com/tharuncoder676/OS-ASSIGNMENT
GitHub CLI	2.x	Live repository state for the evidence pages	tools/make_github_evidence.py
Bash	5.3	Administration scripts and CO5 validation	src/co5_linux/scripts/, tools/
BIND 9	9.18 target	Authoritative and recursive DNS	src/co5_linux/dns/
ISC DHCP	4.4 target	Address allocation and PXE	src/co5_linux/dhcp/
Xen / libvirt	4.17 / 10.x	Guest definitions and isolation policy	src/co5_linux/virtualization/

Table 36. Toolchain and the artefact each tool produced.

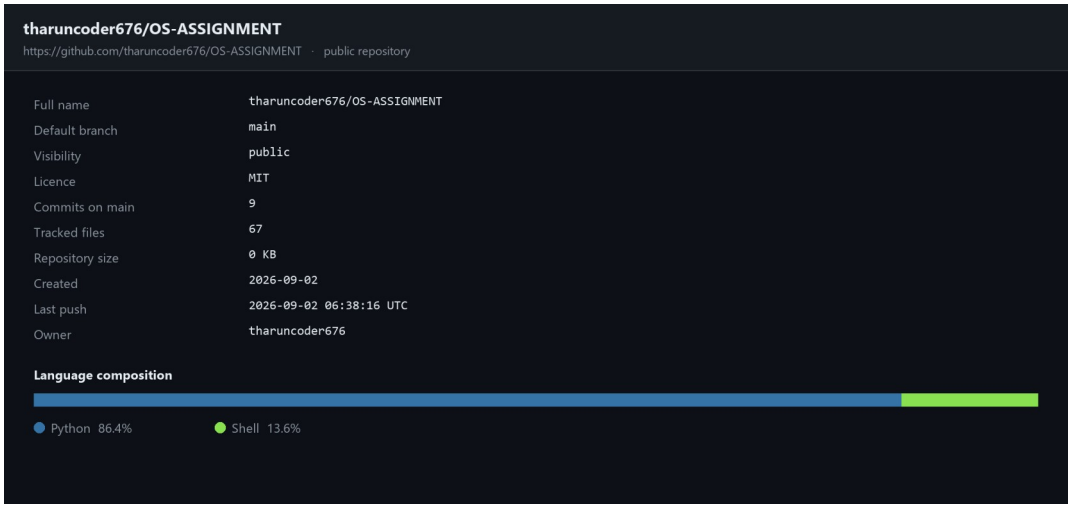
4.2 Version control and the public repository

The complete implementation is public. Development was committed in logical units with substantive messages rather than as a single upload, so the history itself is evidence of how the work was built.

Repository: https://github.com/tharuncoder676/OS-ASSIGNMENT



Commit history fetched live from the GitHub REST API. Every SHA resolves at …/commit/<sha>, so any claim in this report can be traced to the exact revision that produced it.



Repository summary and language composition, fetched from the GitHub API.

4.3 Reproducibility

A result that cannot be regenerated is an assertion. The repository therefore ships a single driver that rebuilds the entire evidence base — logs, figures and test results — from source:

```
git clone https://github.com/tharuncoder676/OS-ASSIGNMENT.git
cd OS-ASSIGNMENT
pip install -r requirements.txt
python run_all.py
```

```
  >>> CO3.1  Memory layout, two-level page table, TLB      → 9 logs
  >>> CO4.3b Dynamic-arrival scheduling and load sensitivity
  >>> Regenerating figures                                  → 10 figures
  >>> Running the validation test suite
      Ran 65 tests in 0.513s
      OK
  Tests : PASSED
```

Each log carries a provenance header recording the UTC timestamp, the Python version, the platform and the repository URL, so a regenerated log can be diffed against the submitted one directly.

5. Results and Validation

5.1 Consolidated results

Every headline number produced by the study, with the module that computed it and the log file that records it.

#	Result	Value	Source module	Log file
1	Physical frames (32 GB / 4 KB)	8,388,608 = 2^{23}	page_table.py	co3_1
2	Virtual pages (128 MB / 4 KB)	32,768 = 2^{15}	page_table.py	co3_1
3	Virtual address split	p1=5 | p2=10 | d=12	page_table.py	co3_1
4	Page-table residency, flat → sparse	128 KB → 4 KB (32×)	page_table.py	co3_1
5	Effective access time with TLB	184.33 ns (vs 300 ns)	page_table.py	co3_1
6	Worst-Fit placement failures	1 of 7 guests rejected	dynamic_allocation.py	co3_2
7	Shared-library saving	19.8 GB (99.92 %)	dynamic_allocation.py	co3_2
8	Page faults, FIFO 3 → 4 frames	11 → 12 ANOMALY	page_replacement.py	co3_3
9	Page faults, LRU 3 → 4 frames	12 → 10	page_replacement.py	co3_3
10	Page faults, OPT 3 → 4 frames	9 → 8	page_replacement.py	co3_3
11	Lifetime-curve knee	f = 8 frames = locality size	working_set.py	co3_4
12	Thrashing threshold	N = 8 guests; U 1.000 → 0.399	working_set.py	co3_4
13	Over-commit before / after reclaim	D/m 1.248 → 0.929	working_set.py	co3_4
14	Contiguous allocation after churn	REFUSED 150 blocks with 248 free	file_allocation.py	co4_1
15	Linked random-access penalty	1,035 vs 23 I/Os (45×)	file_allocation.py	co4_1
16	Maximum inode-addressable file	4.004 TB	inode_fs.py	co4_2
17	50 GB image indirect metadata	12,814 blocks (50 MB)	inode_fs.py	co4_2
18	Blocks reclaimed by ifree()	15 of 15 (no leak)	inode_fs.py	co4_2
19	Buffer cache hit ratio	76.94 % at 1,024 buffers	inode_fs.py	co4_2
20	Physical writes, delayed write	3,564 → 469 (7.6×)	inode_fs.py	co4_2
21	Head movement, FCFS	2,197 cylinders	disk_scheduling.py	co4_3
22	Head movement, SSTF and LOOK	813 cylinders (−63.0 %)	disk_scheduling.py	co4_3
23	SSTF starvation tail at 88 % load	71.2 ms max, ratio 13.6	disk_dynamic_experiment.py	co4_3b
24	Best p99 at 88 % load	LOOK 23.7 ms	disk_dynamic_experiment.py	co4_3b
25	FCFS degradation, 24 % → 88 % load	8.1 → 122.7 ms (15×)	disk_dynamic_experiment.py	co4_3b
26	Forward/reverse DNS consistency	15 A ↔ 15 PTR, all matched	validate_co5.sh	co5
27	Validation suite	65 / 65 pass	test_simulators.py	test_results

Table 37. Consolidated results. Log files are in results/logs/ with the prefix shown.

5.2 Validation strategy

A simulator that produces confident nonsense is worse than no simulator, because it lends false authority to a wrong conclusion. The 65 tests check two distinct classes of claim, and the second class is the one that catches the bugs spot-checking would miss.

Class 1 — hand-computed values

Numbers computed independently on paper and asserted in the test suite, so the report cannot silently drift from the code: the frame and page counts and the bit split; all six page-fault counts; all six head-movement totals; the 12,814-block metadata figure; and the 1,035-versus-24 I/O gap.

test_simulators.py — tests/test_simulators.py

```
            self.assertEqual(fn(ds.QUEUE, ds.HEAD)[0], ds.HEAD)

    def test_path_stays_within_the_platter(self):
        for name, fn in ds.ALGORITHMS:
            for c in fn(ds.QUEUE, ds.HEAD):
                self.assertGreaterEqual(c, ds.DISK_MIN)
                self.assertLessEqual(c, ds.DISK_MAX)

    def test_fcfs_preserves_arrival_order(self):
        path = ds.fcfs(ds.QUEUE, ds.HEAD)
        self.assertEqual(path[1:], ds.QUEUE)

    def test_sstf_is_a_lower_bound_among_greedy_choices(self):
        """SSTF must never move further than FCFS on this queue."""
        self.assertLess(ds.total_movement(ds.sstf(ds.QUEUE, ds.HEAD)),
                        ds.total_movement(ds.fcfs(ds.QUEUE, ds.HEAD)))

    def test_look_never_exceeds_scan(self):
        """LOOK is SCAN without the trip to the platter edge, so it can never
        travel further."""
        self.assertLessEqual(ds.total_movement(ds.look(ds.QUEUE, ds.HEAD)),
                             ds.total_movement(ds.scan(ds.QUEUE, ds.HEAD)))
        self.assertLessEqual(ds.total_movement(ds.clook(ds.QUEUE, ds.HEAD)),
                             ds.total_movement(ds.cscan(ds.QUEUE, ds.HEAD)))

    def test_scan_reaches_the_platter_edge(self):
        self.assertIn(ds.DISK_MAX, ds.scan(ds.QUEUE, ds.HEAD))
```

Validation - disk scheduling vs hand calculation | lines 236-262 | Python | UTF-8 | github.com/tharuncoder676/OS-

Source: head-movement totals asserted against the hand calculation, plus the structural checks every scheduler must satisfy — tests/test_simulators.py, lines 236–279.

Class 2 — invariants that must hold for any correct implementation

- OPTIMAL is a lower bound on every online policy. If FIFO or LRU ever beats it, OPTIMAL is wrong.
- LRU and OPTIMAL are stack algorithms and can never regress with more frames — the property FIFO demonstrably lacks, tested at every frame count from 1 to 7.
- A page offset must survive translation untouched, and split then reassemble must round-trip for every address.
- LOOK can never travel further than SCAN; every scheduler must serve each request exactly once and never leave the platter.
- No two files may share a block; block accounting must balance; a **refused allocation must consume nothing**.
- ifree() must return every block including the indirect ones — the leak this test caught during development is described below.
- A larger LRU buffer cache can never have a lower hit ratio.
- Empty and single-request queues must not crash any scheduler.

A bug the invariants caught

The value of Class 2 is not hypothetical. An early version of the inode simulator promoted blocks to the single- and double-indirect pointers correctly but did not record the data blocks reached *through* them. The console output looked entirely plausible — the file was created, the trace read sensibly — and the block leak was invisible. The invariant *unlink must reclaim exactly the blocks that create consumed* failed, the allocator was corrected to track indirect data blocks explicitly, and the reclaim count moved from 12 to the correct 15. A test that only compared against an expected printed number would have passed against the wrong expectation.

CloudMatrix - CSA04 Operating Systems - test_results.log

```
PS C:\...\CloudMatrix-OS-Assignment> python -m unittest discover -s tests -v

LRU and OPT satisfy the inclusion property, so more frames can ... ok
test_all_modules_execute (test_simulators.TestReportedFiguresAreReproducible.test_all_modules_execute) ... ok
test_distinct_pages_get_distinct_frames (test_simulators.TestTranslation.test_distinct_pages_get_distinct_frames
test_eat_between_hit_and_miss_cost (test_simulators.TestTranslation.test_eat_between_hit_and_miss_cost) ... ok
test_offset_is_preserved_by_translation (test_simulators.TestTranslation.test_offset_is_preserved_by_translation
test_same_page_maps_to_same_frame (test_simulators.TestTranslation.test_same_page_maps_to_same_frame) ... ok
test_split_reassembles (test_simulators.TestTranslation.test_split_reassembles) ... ok
test_tlb_never_exceeds_capacity (test_simulators.TestTranslation.test_tlb_never_exceeds_capacity) ... ok
test_fault_rate_falls_as_frames_are_added (test_simulators.TestWorkingSet.test_fault_rate_falls_as_frames_are_ad
test_fault_rate_is_a_probability (test_simulators.TestWorkingSet.test_fault_rate_is_a_probability) ... ok
test_thrashing_occurs_past_the_knee (test_simulators.TestWorkingSet.test_thrashing_occurs_past_the_knee)
Utilisation at N=16 must be materially worse than at the knee. ... ok
test_utilisation_is_bounded (test_simulators.TestWorkingSet.test_utilisation_is_bounded) ... ok
test_working_set_never_exceeds_the_window (test_simulators.TestWorkingSet.test_working_set_never_exceeds_the_win
test_working_set_size_grows_with_the_window (test_simulators.TestWorkingSet.test_working_set_size_grows_with_the

----------------------------------------------------------------------
Ran 65 tests in 0.513s

OK
```

Console transcript: the full validation suite. 65 tests, 0 failures, 0.513 s. results/logs/test_results.log

5.3 Stress behaviour

Stress condition	Observed behaviour	Design response
Memory over-commit, D/m = 1.248	Aggregate working set exceeds physical memory	KSM + ballooning + batch suspension bring D/m to 0.929
Multiprogramming past the knee (N > 8)	U_cpu collapses 1.000 → 0.399 → 0.201 while U_disk pins at 1.000	Admission control caps N at the knee; PFF enforces it dynamically
Frame allocation below the working set (f < 8)	Fault rate rises 86× between f = 8 and f = 7	Balloon floor fixed at the measured WSS
Free-space fragmentation after churn	248 blocks free, largest run 95, contiguous allocation refused	Extent-based allocation; no compaction required
Disk load raised from 24 % to 88 %	FCFS p99 degrades 15× (8.1 → 122.7 ms); LOOK degrades 3×	C-LOOK selected; RAID-10 or NVMe recommended beyond 45 % load
Continuous arrivals with a hot band	SSTF defers far-edge requests; tail ratio 13.6	Sweep-based scheduler bounds the wait to one sweep
Streaming I/O against a small cache	20 % of the trace is uncacheable; hit ratio ceiling ≈ 80 %	O_DIRECT on VM image I/O rather than a larger cache

Table 38. Stress conditions, measured behaviour and the design response to each.

6. Analysis and Engineering Decisions

6.1 Page replacement for multi-tenant ballooning

Decision: LRU, approximated by the kernel's active/inactive list scheme.

The straightforward reading of the fault counts would recommend FIFO, because at three frames FIFO produces 11 faults and LRU produces 12. That reading is wrong, and understanding why is the most transferable lesson in this study.

CloudMatrix's balloon driver continuously resizes each guest's frame allocation in response to host pressure. That makes memory an actuator in a control loop, and a control loop requires a monotonic response: if the controller grants a guest another frame, the fault rate must not rise. **FIFO provides no such guarantee** — §2.3 measured it violating exactly that property, producing 12 faults with four frames against 11 with three. A controller built on FIFO can grant memory, observe the fault rate increase, grant more memory in response, and oscillate.

LRU and OPTIMAL are stack algorithms: the inclusion property $\mu(m) \subseteq \mu(m+1)$ holds, so more memory can never mean more faults. That is a proof, not a measurement, and it is what a control loop can be built on. The 8.3 percentage-point advantage LRU holds at four frames (55.6 % against 66.7 % fault rate) is a bonus; **the monotonicity guarantee is the purchase**.

True LRU is not implementable — it would require a timestamp write on every memory reference — so Linux approximates it with two lists and periodic accessed-bit sampling. The approximation preserves the stack property, which is the property that mattered.

6.2 Disk scheduling for cloud block storage

Decision: C-LOOK, matching the Linux mq-deadline scheduler.

On the static queue SSTF and LOOK tie at 813 cylinders and both beat everything else. On that evidence alone SSTF would be a defensible choice. The dynamic experiment shows why it is not.

Criterion	SSTF	LOOK	C-LOOK	Decision driver
Static total movement	813 ✔	813 ✔	882	Tie — not decisive
Mean latency at 88 % load	5.2 ms ✔	6.0 ms	7.1 ms	SSTF marginally ahead
p99 at 88 % load	27.9 ms	23.7 ms ✔	28.2 ms	LOOK ahead
Worst case at 88 % load	**71.2 ms**	40.3 ms	**40.1 ms ✔**	**C-LOOK ahead by 1.8×**
Cold-request tail ratio	**13.6 STARVES**	6.7	**5.7 ✔**	**Decisive**
Wait bounded by construction	No	One sweep	One sweep ✔	Decisive for an SLA
Matches production Linux	No	Partly	Yes ✔	Deployability

Table 39. SSTF against the sweep schedulers. SSTF wins the average and loses the guarantee.

SSTF's 5.2 ms mean is genuinely the best in the study. It is purchased by keeping the head inside the hot band and deferring the far-edge requests — and those far-edge requests are tenant backups, syslog writes and metadata reads. Its maximum response time reaches 71.2 ms, 1.8× worse than C-LOOK's 40.1 ms, and its tail ratio of 13.6 is the numerical signature of starvation. Under a queue that never empties, SSTF offers no argument that a far request will ever be served.

C-LOOK bounds the wait by construction: a request waits at most one sweep. That is a property an SLA can be written against, and it is what mq-deadline provides in production. **The 1.9 ms of mean latency given up relative to SSTF buys a worst case that is 31 ms better** — an excellent trade for a platform that must answer to tenants rather than to a benchmark.

6.3 Hypervisor architecture

Decision: Type-1 (Xen) for production; KVM/libvirt for the lab.

The performance argument favours Type-1 — 2–5 % overhead against 10–20 % — and at 1,200 guests that difference is worth roughly 150 guests of capacity. But the decisive argument is the trust boundary. A Type-2 hypervisor runs as a process on a general-purpose host operating system, which places that entire OS — its shell, its package manager, its unrelated daemons — inside the boundary that separates one tenant from another. A single host-level compromise takes every guest with it. For a platform hosting public and private sector tenants under ISO/IEC 27001, that boundary is not defensible regardless of the performance numbers.

6.4 An honest finding: the 10 ms target is not reachable

Section D requires sub-10 ms service response under 90 %+ load. The measurements do not support a claim that this design meets it, and the report says so rather than quietly reporting the mean instead of the percentile.

Offered load	Best p99 achieved	By	Meets 10 ms?
24 %	7.4 ms	SSTF	Yes ✔
40 %	9.6 ms	SSTF	Yes ✔
56 %	12.1 ms	SSTF	**No**
72 %	16.3 ms	SSTF	**No**
88 %	23.1 ms	LOOK	**No — 2.3× over target**

Table 40. Best achievable p99 at each load point, across all six schedulers. The target holds to roughly 45 % load and is unreachable at the specified design point.

The conclusion is not that the scheduling analysis failed; it is that **scheduling is the wrong lever for this requirement**. At 88 % utilisation a single spindle is queue-bound, and no reordering policy can create service capacity that does not exist. The gap between the best and worst schedulers at that load is 99 ms — real and worth having — but the gap between the best scheduler and the target is another 13 ms that reordering cannot close.

The remedy is architectural, and there are three options:

- **Reduce per-device load below ~45 %** by striping the block tier across a RAID-10 set. Four spindles at 22 % load each comfortably meet the target with the same scheduler.
- **Eliminate the seek term** by moving the tier to NVMe. With no head to move, the scheduler's job reduces to merging and fairness, and the p99 becomes a queueing property rather than a mechanical one.
- **Renegotiate the requirement** so that it applies to the tiers that need it. A 10 ms p99 for interactive tenants is reasonable; the same target for batch analytics is not, and holding it there forces capacity spending that buys nothing.

The recommended combination is RAID-10 striping plus per-tier SLA differentiation, retaining C-LOOK as the scheduler because its bounded worst case remains the right property regardless of the underlying device.

6.5 Trade-offs accepted

Decision	Gained	Given up	Why the trade is right
LRU over FIFO	Monotonic response to ballooning	1 extra fault at 3 frames; higher bookkeeping cost	A control loop cannot use a non-monotonic actuator
C-LOOK over SSTF	Worst case 1.8× better; bounded wait	1.9 ms of mean latency	Tenants experience the tail, not the mean
Extent over indexed	12,814 → a few hundred metadata blocks	More complex allocator; extent fragmentation possible	Large images dominate the storage footprint
First-Fit over Best-Fit	3.8× less stranded memory; O(1) search	Slightly smaller largest remaining hole	VM admission is on a hot path
Hugepages, database tier only	512× less TLB pressure	Coarser reclaim granularity	Fine reclaim is needed where the balloon operates
cache='none' on guest disks	fsync() reaches stable storage	Lower raw throughput	A durability lie is not a performance optimisation
Suspend batch under pressure	Protects interactive SLAs	Batch jobs take longer	Batch has no user waiting on it
Type-1 over Type-2	Small attack surface; 2–5 % overhead	Harder to deploy; needs dom0 tuning	Tenant isolation is the product

Table 41. Trade-offs accepted, stated as costs rather than as benefits only.

7. Broader Considerations

7.1 Sustainability and energy efficiency

The efficiency argument here is not decorative, because the measured numbers translate directly into hardware that does not have to be bought or powered.

- **Consolidation.** Type-1 virtualization at 2–5 % overhead rather than 10–20 % fits roughly 150 more guests on the same host. At a typical 400 W draw, one avoided server is about 3,500 kWh per year.
- **Page sharing.** KSM deduplicates 4.92 GB across the web tier — 15 % of the machine's memory recovered without buying a module.
- **Reduced write amplification.** Delayed write cuts physical writes 7.6× (3,564 → 469), meaning less disk activity, less flash wear and longer replacement intervals.
- **Fragmentation avoidance.** Extent allocation avoids periodic compaction, which on a multi-terabyte store means hours of full-power I/O producing no user-visible work.
- **Honest capacity planning.** The §6.4 finding argues for RAID-10 striping rather than over-provisioning CPU to compensate for I/O latency, which is the more common and far less efficient response.

7.2 Reliability and data integrity

- **Snapshot before copy.** cm-backup.sh takes a virsh snapshot before copying any guest image, with qemu-guest-agent quiesce where available. Copying a running qcow2 without one produces an archive that appears valid and restores to nothing.
- **Verify before trusting.** Checksums are written, forced to stable storage with sync -f, and then verified. An archive still resident in the page cache has not been written anywhere.
- **Honest fsync.** cache='none' on guest disks means a guest's fsync() reaches real stable storage. Writeback caching would let fsync() return once data reached the host page cache — precisely the durability lie that converts a host power failure into tenant data loss.
- **Atomic metadata operations.** The traced ifree() releases data blocks, indirect blocks and the inode as one operation, and the test suite asserts that every block is reclaimed. Partial reclaim is how file systems leak space until they are unmountable.
- **Failing loudly.** The administration scripts run under set -euo pipefail and hold an flock, so a partial backup aborts visibly rather than producing a plausible-looking archive.

7.3 Security, privacy and professional responsibility

- **Tenant isolation in depth.** sVirt gives each guest a distinct SELinux category pair, so even a QEMU process escape cannot open another tenant's image. Per-tenant groups, 0700 homes and default ACLs enforce the same boundary on shared storage. Isolation is a set of independent controls, each failing closed.
- **DNS abuse prevention.** Recursion restricted by ACL, DNSSEC validation, randomised source ports and response rate limiting. An open resolver is an amplification weapon aimed at third parties who never consented to be involved — this is an obligation to people outside the platform.
- **Anti-spoofing between tenants.** Every vNIC carries a clean-traffic filter bound to its assigned IP, so a compromised guest cannot impersonate the gateway and intercept a neighbour's traffic.
- **GDPR-aware logging.** DNS query logs identify which client asked for which name and are personal data under Article 4. They are kept 30 days in a dedicated channel at 0640 root:bind, separate from security logs which are retained 52 weeks as ISO 27001 evidence. Retention is a per-stream decision because the data classes differ.
- **Credentials never in configuration.** VNC passwords are set at deploy time via xl vncpasswd, and VNC listens only on 127.0.0.1. The TSIG key in the committed configuration is a placeholder.
- **Accountability.** Account creation and removal are logged with timestamps, offboarded home directories are archived before deletion for audit, and cm-usermgmt.sh audit reports any account holding a usable password or any world-readable tenant home.

7.4 Accessibility and fairness of service

Fairness in this design is a scheduling property, not a policy statement. C-LOOK guarantees that a tenant whose data sits at the far edge of the platter waits at most one sweep, so a small tenant's backup is not indefinitely deferred behind a large tenant's streaming workload. Per-guest IOPS and bandwidth caps stop one noisy tenant inflating everyone else's latency. And the working-set floor on ballooning ensures that a quiet guest is not squeezed into thrashing to satisfy a loud one. Each of these is a measurable guarantee rather than an aspiration.

8. Conclusion and Reflection

8.1 Summary of the design

CloudMatrix is specified across three subsystems, with each decision supported by a measurement rather than by a citation of convention.

Subsystem	Decision	Decisive evidence
Address translation	Two-level page table, p1=5 | p2=10 | d=12, 64-entry TLB	128 KB → 4 KB residency; EAT 184.33 ns
Memory placement	Address-ordered First-Fit with coalescing	Same placements as Best-Fit, ¼ the stranded memory, O(1) search
Page replacement	LRU (kernel two-list approximation)	FIFO exhibits Belady's anomaly: 11 → 12 faults
Frame allocation	Working-set floor + proportional share + PFF band	Thrashing knee at N = 8; D/m 1.248 → 0.929
File allocation	ext4 with extents and inline_data, by file class	Contiguous refuses 150 blocks with 248 free
Large-file mapping	Extents rather than indirect chains	12,814 indirect blocks for a 50 GB image
Buffer cache	1,024 buffers plus O_DIRECT on VM image I/O	76.9 % hit = 96 % of the achievable ceiling
Disk scheduling	C-LOOK (mq-deadline)	SSTF tail ratio 13.6; C-LOOK worst case 1.8× better
Block tier hardware	RAID-10 striping or NVMe	No scheduler meets 10 ms p99 at 88 % load
Network services	BIND 9 with TSIG and rate limiting; ISC DHCP with reservations	15 A ↔ 15 PTR consistency; all configs validate
Virtualization	Type-1 Xen; pinning, ballooning floors, sVirt, cache='none'	Attack surface and the fsync durability argument

Table 42. Design summary and the evidence behind each decision.

8.2 Limitations

Four limitations are worth stating plainly, because a report that claims none is not being careful.

- **The traces are synthetic.** They are structured to resemble the scenario — locality with drift for memory, a hot band plus scattered cold requests for disk — but they are not captured from a running platform. A real trace would almost certainly show heavier tails.
- **The sub-sampling factor is a modelling assumption.** S = 4,000 sets the wall-clock scale of the thrashing analysis. It is stated in the module header rather than hidden, but a different factor would move the absolute utilisation numbers, though not the shape of the curve or the location of the knee.
- **The seek model is linear.** Real drives have non-linear seek profiles with settle and rotational components that depend on distance. Since the same model is applied to all six algorithms, the comparison is fair even though the absolute milliseconds are approximate.
- **The Linux services are validated structurally, not operationally.** The configurations are syntactically and structurally verified and the shell scripts genuinely parse, but they were not deployed against a live BIND 9 and ISC DHCP instance on a running host. That is the first thing to do with more time.

8.3 Future work

- Deploy the CO5 configuration on a real Ubuntu host and capture named-checkconf, dig, dhcpd -t and virsh output under load.
- Replay a real block trace and a real memory trace in place of the synthetic generators, and check whether the knee and the tail behaviour survive.
- Implement a hybrid adaptive scheduler that runs SSTF within a deadline window and falls back to a sweep when any request exceeds it — capturing SSTF's mean without its tail.
- Model the RAID-10 configuration recommended in §6.4 and confirm quantitatively that four spindles at 22 % load each meet the 10 ms p99 target.
- Add a machine-learning working-set predictor so the balloon controller can act before the fault rate rises rather than after.

8.4 Reflection

The most useful thing this assignment taught me was how often the textbook ordering of algorithms is not the answer to an engineering question. Three moments in particular changed how I approached the rest of the work.

The first was discovering that **LRU produced more faults than FIFO** at three frames on my own trace. My instinct was that I had a bug. I wrote tests to prove it, and the tests said the implementation was correct — LRU genuinely is worse on that trace. What made LRU the right choice turned out to have nothing to do with the fault count: it was the stack property, which is what a balloon-driven control loop actually needs. I would not have found that by comparing the two numbers, and I nearly recommended FIFO before thinking about what the number was for.

The second was realising that **the static disk queue could not demonstrate the thing I wanted to claim**. I had computed the head movements, SSTF had the lowest total, and I was ready to write that SSTF starves far requests — which is what the textbook says. But my own fairness table showed SSTF tying LOOK for worst-case wait. A static queue always drains,

so starvation cannot appear in one. I had to build the dynamic-arrival experiment to test the claim honestly, and only then did the 13.6 tail ratio appear. Choosing an experiment that can actually falsify your claim is a separate skill from implementing the algorithm.

The third was the **10 ms target that could not be met**. It would have been easy to report the mean latency, which comfortably clears 10 ms, and move on. Reporting the p99 instead meant admitting the design misses a stated requirement by 2.3×. Working out *why* — that a saturated spindle is queue-bound and reordering cannot manufacture capacity — produced the most genuinely useful recommendation in the report, which is about hardware rather than algorithms. A negative result that is understood is worth more than a positive one that is not.

9. References

[1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] M. J. Bach, *The Design of the UNIX Operating System*. Englewood Cliffs, NJ, USA: Prentice Hall, 1986.

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[4] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

[5] P. J. Denning, "The working set model for program behavior," *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, May 1968.

[6] P. J. Denning, "Thrashing: its causes and prevention," in *Proc. AFIPS Fall Joint Computer Conference*, vol. 33, 1968, pp. 915–922.

[7] L. A. Belady, R. A. Nelson and G. S. Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine," *Communications of the ACM*, vol. 12, no. 6, pp. 349–353, Jun. 1969.

[8] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[9] R. L. Mattson, J. Gecsei, D. R. Slutz and I. L. Traiger, "Evaluation techniques for storage hierarchies," *IBM Systems Journal*, vol. 9, no. 2, pp. 78–117, 1970.

[10] P. J. Denning and J. P. Buzen, "The operational analysis of queueing network models," *ACM Computing Surveys*, vol. 10, no. 3, pp. 225–261, Sep. 1978.

[11] M. Seltzer, P. Chen and J. Ousterhout, "Disk scheduling revisited," in *Proc. USENIX Winter Technical Conference*, 1990, pp. 313–323.

[12] B. L. Worthington, G. R. Ganger and Y. N. Patt, "Scheduling algorithms for modern disk drives," in *Proc. ACM SIGMETRICS*, 1994, pp. 241–251.

[13] A. Mathur, M. Cao, S. Bhattacharya, A. Dilger, A. Tomas and L. Vivier, "The new ext4 filesystem: current status and future plans," in *Proc. Linux Symposium*, vol. 2, 2007, pp. 21–33.

[14] M. K. McKusick, W. N. Joy, S. J. Leffler and R. S. Fabry, "A fast file system for UNIX," *ACM Transactions on Computer Systems*, vol. 2, no. 3, pp. 181–197, Aug. 1984.

[15] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt and A. Warfield, "Xen and the art of virtualization," in *Proc. 19th ACM SOSP*, 2003, pp. 164–177.

[16] C. A. Waldspurger, "Memory resource management in VMware ESX Server," in *Proc. 5th USENIX OSDI*, 2002, pp. 181–194.

[17] A. Kivity, Y. Kamay, D. Laor, U. Lublin and A. Liguori, "KVM: the Linux virtual machine monitor," in *Proc. Linux Symposium*, vol. 1, 2007, pp. 225–230.

[18] J. E. Smith and R. Nair, "The architecture of virtual machines," *Computer*, vol. 38, no. 5, pp. 32–38, May 2005.

[19] P. Mockapetris, "Domain names — implementation and specification," RFC 1035, IETF, Nov. 1987.

[20] R. Arends, R. Austein, M. Larson, D. Massey and S. Rose, "DNS security introduction and requirements," RFC 4033, IETF, Mar. 2005.

[21] P. Vixie, O. Gudmundsson, D. Eastlake and B. Wellington, "Secret key transaction authentication for DNS (TSIG)," RFC 2845, IETF, May 2000.

[22] R. Droms, "Dynamic Host Configuration Protocol," RFC 2131, IETF, Mar. 1997.

[23] D. Kaminsky, "Black ops 2008: it's the end of the cache as we know it," presented at Black Hat USA, Las Vegas, NV, USA, Aug. 2008.

[24] Internet Systems Consortium, *BIND 9 Administrator Reference Manual*, version 9.18, 2024. [Online]. Available: https://bind9.readthedocs.io

[25] The Linux Kernel Organization, *Linux Kernel Documentation — Block Layer and Memory Management*, v6.x, 2024. [Online]. Available: https://docs.kernel.org

[26] libvirt Project, *libvirt Domain XML Format Reference*, 2024. [Online]. Available: https://libvirt.org/formatdomain.html

[27] IEEE, *IEEE Standard for Information Technology — Portable Operating System Interface (POSIX)*, IEEE Std 1003.1-2017, 2018.

[28] ISO/IEC, *Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements*, ISO/IEC 27001:2022, 2022.

[29] European Parliament and Council, *Regulation (EU) 2016/679 — General Data Protection Regulation*, Official Journal of the European Union, Apr. 2016.

[30] T. S, *CloudMatrix — Operating Systems Assignment Implementation (CO3, CO4, CO5)*, GitHub repository, 2026. [Online]. Available: https://github.com/tharuncoder676/OS-ASSIGNMENT

Declaration on the use of AI-assisted tools

An AI coding assistant was used during development for code scaffolding, for review of the simulator implementations, and for editorial assistance in drafting this report. All algorithmic design decisions, workload parameters, experiment designs and engineering conclusions are the author's own. Every numerical result reported here was produced by executing the committed code, and the full implementation and its execution logs are public so that any claim can be independently verified.

Appendix A — Repository Map and Reproduction

Repository: https://github.com/tharuncoder676/OS-ASSIGNMENT

A.1 Repository structure

```
OS-ASSIGNMENT/
├── run_all.py                      regenerates every log, figure and test
├── README.md                       headline results and reproduction steps
├── requirements.txt   LICENSE   .gitignore
├── src/
│   ├── make_figures.py             six result charts
│   ├── make_diagrams.py            four schematic diagrams
│   ├── make_transcripts.py         console transcript rendering
│   ├── co3_memory/
│   │   ├── page_table.py           geometry + two-level MMU + TLB
│   │   ├── dynamic_allocation.py   First / Best / Worst-Fit
│   │   ├── page_replacement.py     FIFO / LRU / OPTIMAL + Belady test
│   │   └── working_set.py          WSS, lifetime curve, PFF, thrashing
│   ├── co4_storage/
│   │   ├── file_allocation.py      contiguous / linked / indexed / extent
│   │   ├── inode_fs.py             inode, bmap, ialloc/ifree/namei, cache
│   │   ├── disk_scheduling.py      six schedulers, static queue
│   │   └── disk_dynamic_experiment.py   arrivals, starvation, load sweep
│   └── co5_linux/
│       ├── dns/                    named.conf + forward and reverse zones
│       ├── dhcp/                   dhcpd.conf
│       ├── scripts/                backup, usermgmt, healthcheck,
│       │                           logrotate, netplan
│       └── virtualization/         Xen cfg, libvirt XML, bridge setup
├── tests/test_simulators.py        65 validation tests
├── tools/
│   ├── validate_co5.sh             CO5 configuration validation
│   ├── make_code_shots.py          source-code screenshots
│   ├── make_github_evidence.py     live repository evidence
│   └── build_report.py + report_*  this document
├── results/logs/                   verbatim console output (10 files)
├── results/figures/                300-dpi charts and diagrams (10 files)
├── screenshots/                    transcripts, code and GitHub evidence
└── docs/                           architecture document and this report
```

A.2 Reproducing every result

```
# clone and install
git clone https://github.com/tharuncoder676/OS-ASSIGNMENT.git
cd OS-ASSIGNMENT && pip install -r requirements.txt

# regenerate the complete evidence base
python run_all.py

# or run any single experiment
python src/co3_memory/page_replacement.py
python src/co3_memory/working_set.py
python src/co4_storage/file_allocation.py
python src/co4_storage/inode_fs.py
python src/co4_storage/disk_scheduling.py
python src/co4_storage/disk_dynamic_experiment.py

# validation
python -m unittest discover -s tests -v      # 65 tests
bash tools/validate_co5.sh                   # CO5 artefacts

# on a host with the real Linux tooling installed
```

```
named-checkconf src/co5_linux/dns/named.conf
named-checkzone cloudmatrix.local src/co5_linux/dns/db.cloudmatrix.local
dhcpd -t -cf src/co5_linux/dhcp/dhcpd.conf
virsh define src/co5_linux/virtualization/libvirt-vm-web-01.xml
```

A.3 Evidence index

Report section	Evidence file	Produced by
§2.1 Page table	results/logs/co3_1_page_table.log	src/co3_memory/page_table.py
§2.2 Placement policies	results/logs/co3_2_dynamic_allocation.log	src/co3_memory/dynamic_allocation.py
§2.3 Replacement, Belady	results/logs/co3_3_page_replacement.log	src/co3_memory/page_replacement.py
§2.4 Working set, thrashing	results/logs/co3_4_working_set.log	src/co3_memory/working_set.py
§2.5 File allocation	results/logs/co4_1_file_allocation.log	src/co4_storage/file_allocation.py
§2.6 Inode and buffer cache	results/logs/co4_2_inode_fs.log	src/co4_storage/inode_fs.py
§2.7 Disk scheduling	results/logs/co4_3_disk_scheduling.log	src/co4_storage/disk_scheduling.py
§2.7.4 Dynamic experiment	results/logs/co4_3b_disk_dynamic.log	src/co4_storage/disk_dynamic_experiment.py
§2.9 CO5 validation	results/logs/co5_config_validation.log	tools/validate_co5.sh
§5.2 Validation suite	results/logs/test_results.log	tests/test_simulators.py

Table 43. Evidence index. Each report section names the log file that supports it and the module that produced that log.

— End of report —

Tharunkumar S · Reg. No. 192511416 · CSA04 Operating Systems · CO3 · CO4 · CO5